COPYRIGHT WARNING: Copyright in these original lectures is owned by Monash University. You may transcribe, take notes, download or stream lectures for the purpose of your research and study only. If used for any other purpose, (excluding exceptions in the Copyright Act 1969 (Cth)) the University may take legal action for infringement of copyright.

Do not share, redistribute, or upload the lecture to a third party without a written permission!

FIT3181/5215 Deep Learning

Week 07: DL for time-series and temporal data- RNNs and LSTMs

CE/Lecturer: Dr Trung Le (Clayton)

Lecturer: Dr Binh Nguyen (Clayton)

Lecturer: Dr Ting Fung Fung (Malaysia)

Lecturer: Dr Arghya Pal (Malaysia)

Lecturer: Dr Sailaja Rajanala (Malaysia)

Email: trunglm@monash.edu

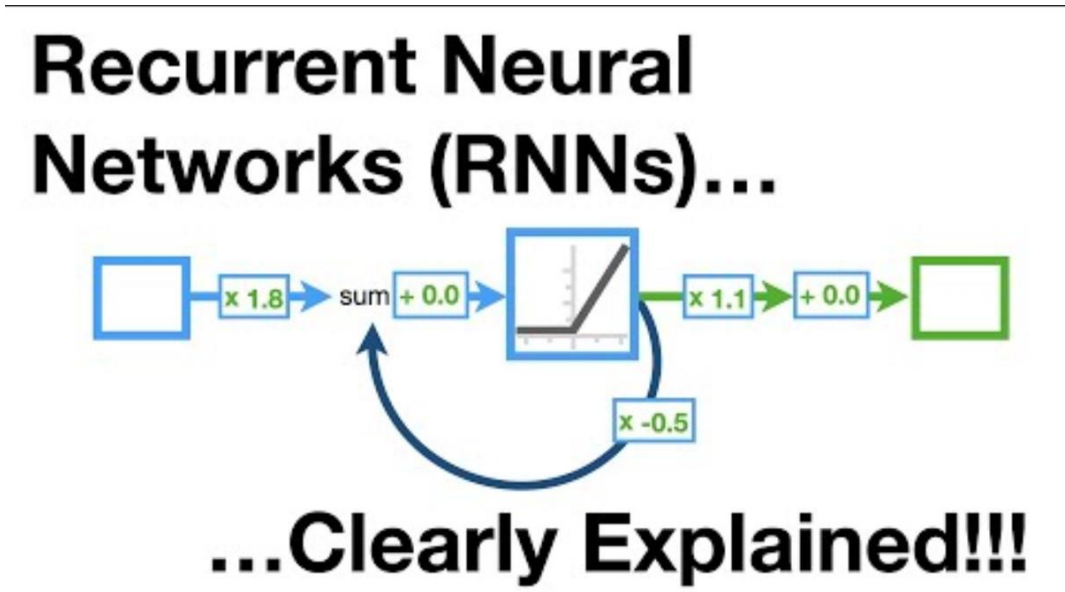


GROUP OF EIGHT AUSTRALIA

Department of Data Science and AI
Faculty of Information Technology, Monash University, Australia

Outline

- Introduction of sequential data
- Recurrent neural networks (RNNs)
- Memory cells
  - Long-short term memory (LSTM) cell
  - Gated recurrent unit (GRU) cell
- Applications of RNNs
  - Sentiment analysis
  - Text generation



Sequence modelling

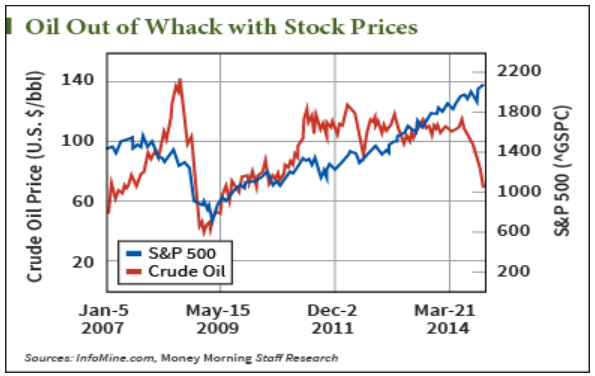
Time series and sequential data



Video surveillance



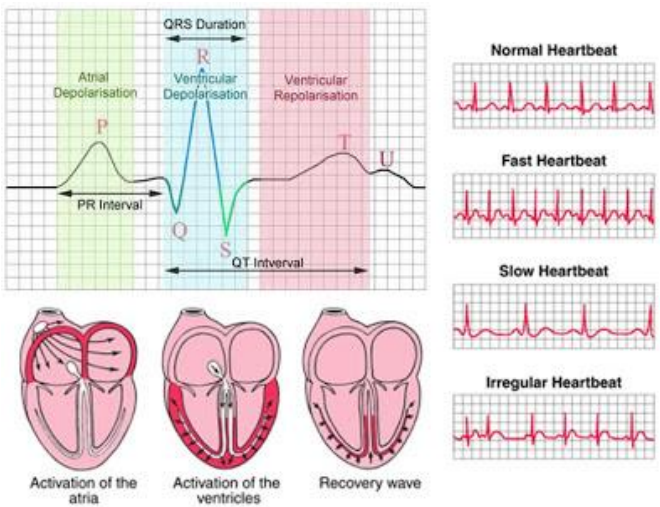




Stock market prices

- We live in a **time-space universe**
- All data collected has a **timestamp**
- Time-series/sequential data

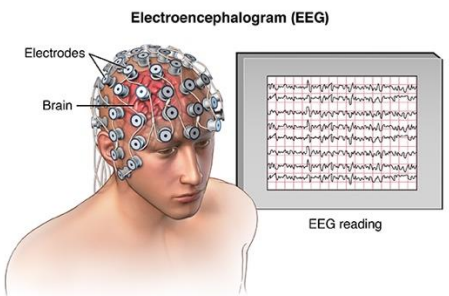
  = collection of sequential data points indexed by time order!



Electrocardiography signals = electrical activity of the heart over time



Texts, messages from media



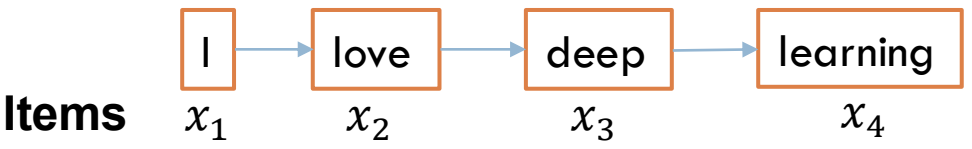
EEG brain signal

Sequential data examples

- Data can also be viewed from different, more subtle angles

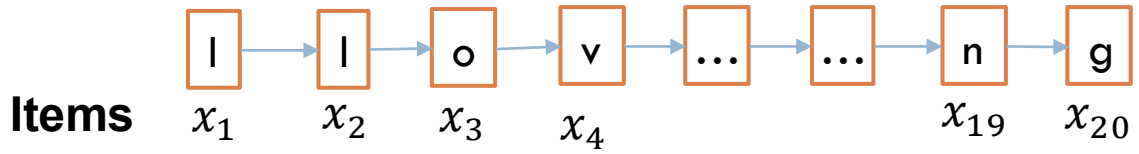
I love deep learning

- **Word level**
  - I, love, deep, learning
- **Character level**
  - I, l, o, v, e, d, e, e, p, l, e, a, r, n, i, n, g



How words/items x_1, x_2, x_3, x_4 **cooccur** in a meaningful text?

$$p(x_1, x_2, x_3, x_4) = p(x_1)p(x_2 \mid x_1)p(x_3 \mid x_1, x_2)p(x_4 \mid x_1, x_2, x_3)$$
$$= p(x_1)p(x_2 \mid x_1)p(x_3 \mid x_{1:2})p(x_4 \mid x_{1:3})$$

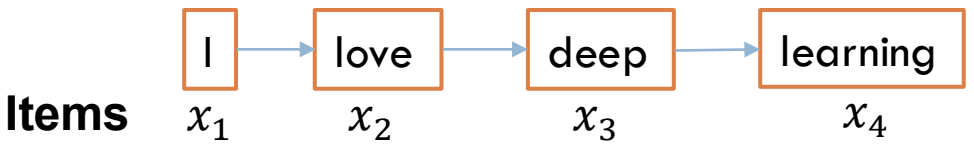


Sequential data examples

- Data can also be viewed from different, more subtle angles

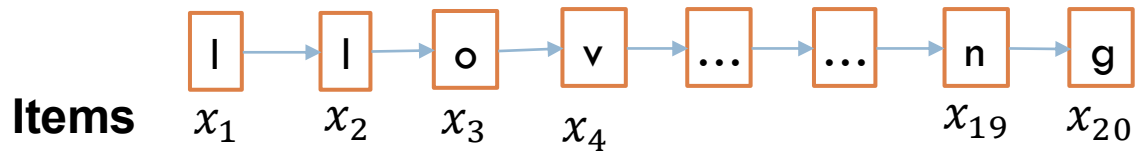
I love deep learning

- **Word level**
  - I, love, deep, learning
- **Character level**
  - I, l, o, v, e, d, e, e, p, l, e, a, r, n, i, n, g



How words/items x_1, x_2, x_3, x_4 **cooccur** in a meaningful text?

$$p(x_1, x_2, x_3, x_4) = p(x_1)p(x_2 \mid x_1)p(x_3 \mid x_1, x_2)p(x_4 \mid x_1, x_2, x_3)$$
$$= p(x_1)p(x_2 \mid x_1)p(x_3 \mid x_{1:2})p(x_4 \mid x_{1:3})$$

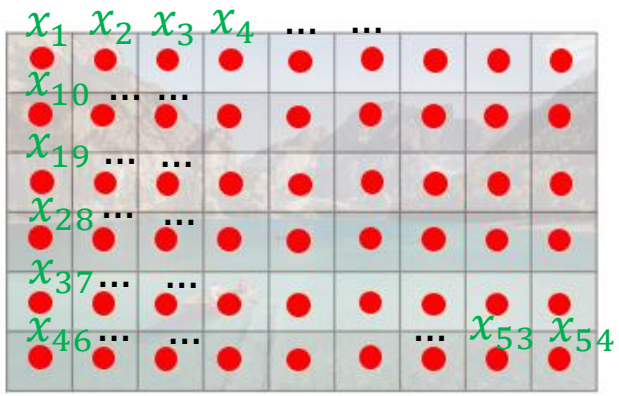


$$p(x_1, x_2, \ldots, x_{20}) = p(x_1)p(x_2 \mid x_1)p(x_3 \mid x_{1:2})p(x_4 \mid x_{1:3}) \ldots p(x_{19} \mid x_{1:18})p(x_{20} \mid x_{1:19})$$

Sequential data examples

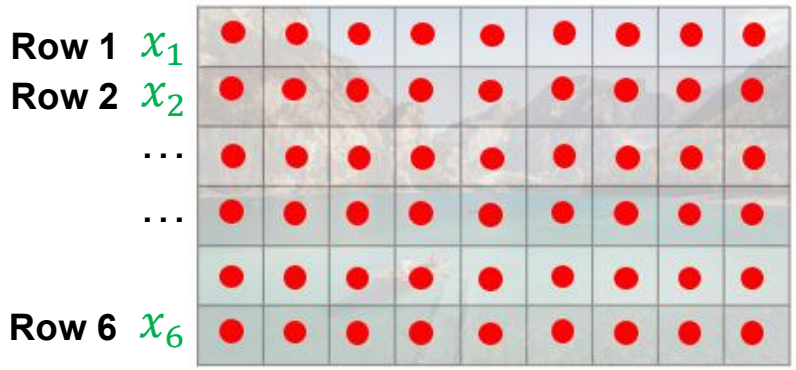


- Sequence of pixels or rows of pixels



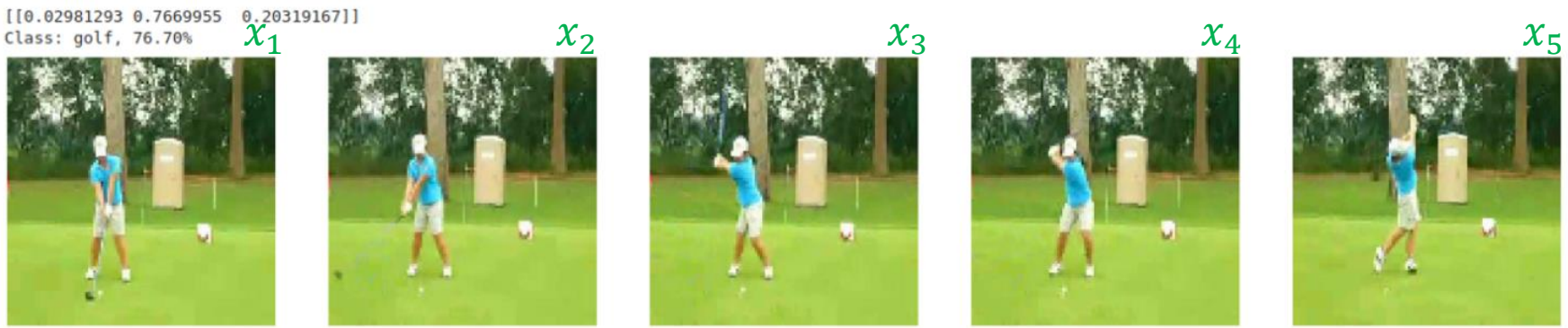
- Sequence of pixels

$$p(x_{1:54}) = p(x_1)p(x_2 \mid x_1) \dots p(x_{54} \mid x_{1:53})$$



- Sequence of rows

$$p(x_{1:6}) = p(x_1)p(x_2 \mid x_1) \dots p(x_6 \mid x_{1:5})$$



- Video as a **sequence of frames/images** (Source: medium.com)

$$p(x_1, \dots, x_5) = p(x_1)p(x_2 \mid x_1)p(x_3 \mid x_{1:2})p(x_4 \mid x_{1:3})p(x_5 \mid x_{1:4})$$

Time series data

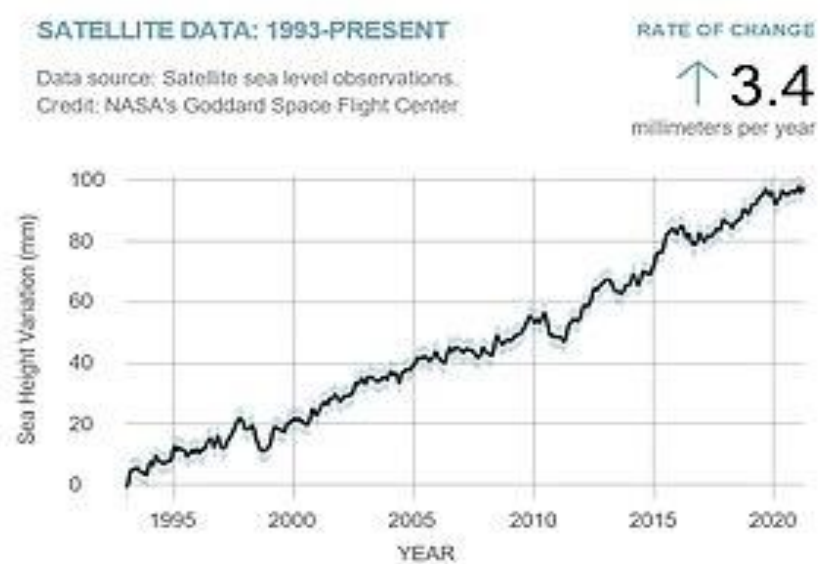


$$p(x_{1:10}) = p(x_1)p(x_2 \mid x_1) \dots p(x_{10} \mid x_{1:9})$$

- Too simple and naïve modelling (i.e., underfitting modelling)
- Historical data are **not sufficient**
- Many **external factors**
  - Human behaviour, the success of Tesla, climate change, weather, and so on

$$p(x_t \mid x_{t-1}, \dots, x_{t-k}) = ???$$

- If we consider the **stock prices of the last k years**, what the **stock price** of the **current year**?



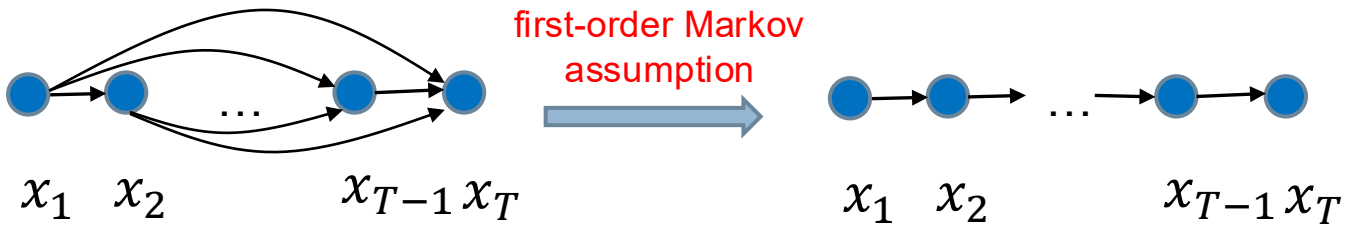
$$p(x_{1995:2020}) = p(x_{1995})p(x_{1996} \mid x_{1995}) \dots p(x_{2020} \mid x_{1995:2019})$$

x_i is **sea level rise** in year i

$$p(x_t \mid x_{t-1}, \dots, x_{t-k}) = ???$$

How to model sequential data?

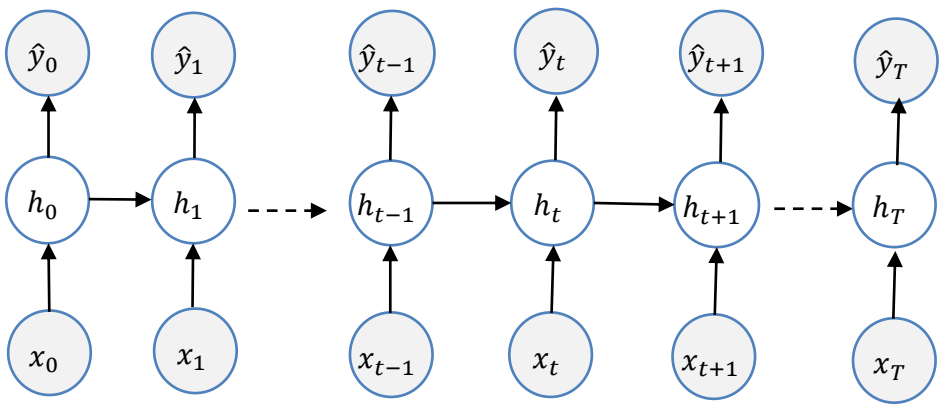
- **Markov models**
  - Hidden Markov Models (HMM), Dynamic Bayesian Networks (DBN), etc.



$p(x_1,..,x_T)$
$= p(x_1)p(x_2 \mid x_1)p(x_3 \mid x_{1:2}) \dots p(x_T \mid x_{1:T-1})$

$p(x_1,..,x_T)$
$= p(x_1)p(x_2 \mid x_1)p(x_3 \mid x_2) \dots p(x_T \mid x_{T-1})$

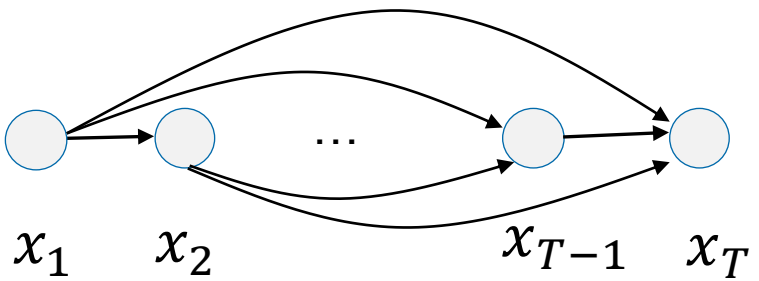
- **Autoregressive model**



- The hidden state h_t stores **information of past observations $x_{0:t}$ = lossy historical summary**
- h_t is used to predict $\hat{y}_t$.
- $h_t = g(h_{t-1}, x_t)$ depends on $\boldsymbol{h_{t-1}}$ and $\boldsymbol{x_t}$.
- $p(x_0,..,x_T) = p(x_0)p(x_1 \mid x_0)p(x_2 \mid x_{0:1}) \dots p(x_T \mid x$ $h_{T-1})$
- RNNs belong to the family of autoregressive models

What are time-series and sequential data?

- Time-series/sequential data = collection of sequential data points indexed by time order
  - Traditionally, it is often the collection of measurements recorded repeatedly over time for the same object of interest
    - e.g., stock market prices, position of the missile, location of the car
  - However, modern machine learning problems often deal with timestamped data collected from heterogeneous sources:
    - e.g., analysing stock market prices together with real-world events, fusion of missile location bearings + weather information for object tracking
    - DL revolution has enabled much more sophisticated/complex problems
- What to model?
  - When historical observations influence on the future



The central question is to model the joint distribution $p(x_1, x_2, \ldots, x_T)$

Sequence Modelling: Summary

- Pre-Deep Learning Models:
  - Typically exploit or make assumption on the model structures to facilitate inference and model training.
  - Hidden Markov Models (discrete state)
    - Factorial Hidden Markov Models, Coupled HMM, Hierarchical HMMs
  - State-space models (Kalman filters, continuous states):
    - Hidden state is a continuous random variable, usually with some Gaussian assumptions
  - Dynamic Bayesian Networks
  - Conditional Random Fields (CRFs), etc

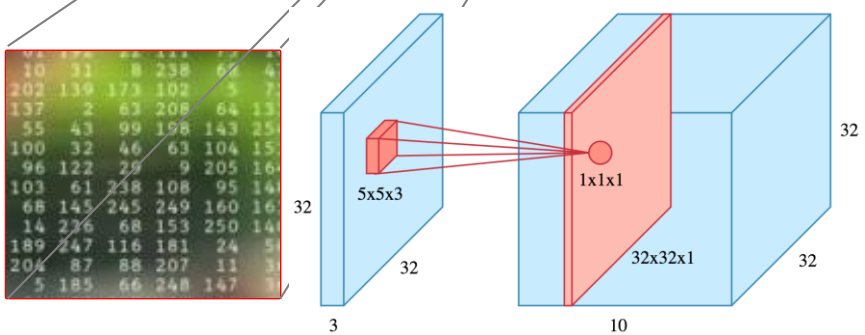
- Deep learning:
  - Leverage on the power of DNNs to capture time-varying dependency
  - Autoregressive models, Deep Recurrent Neural Networks
  - Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU)
  - Seq2Seq models, memory networks, transformer, etc.

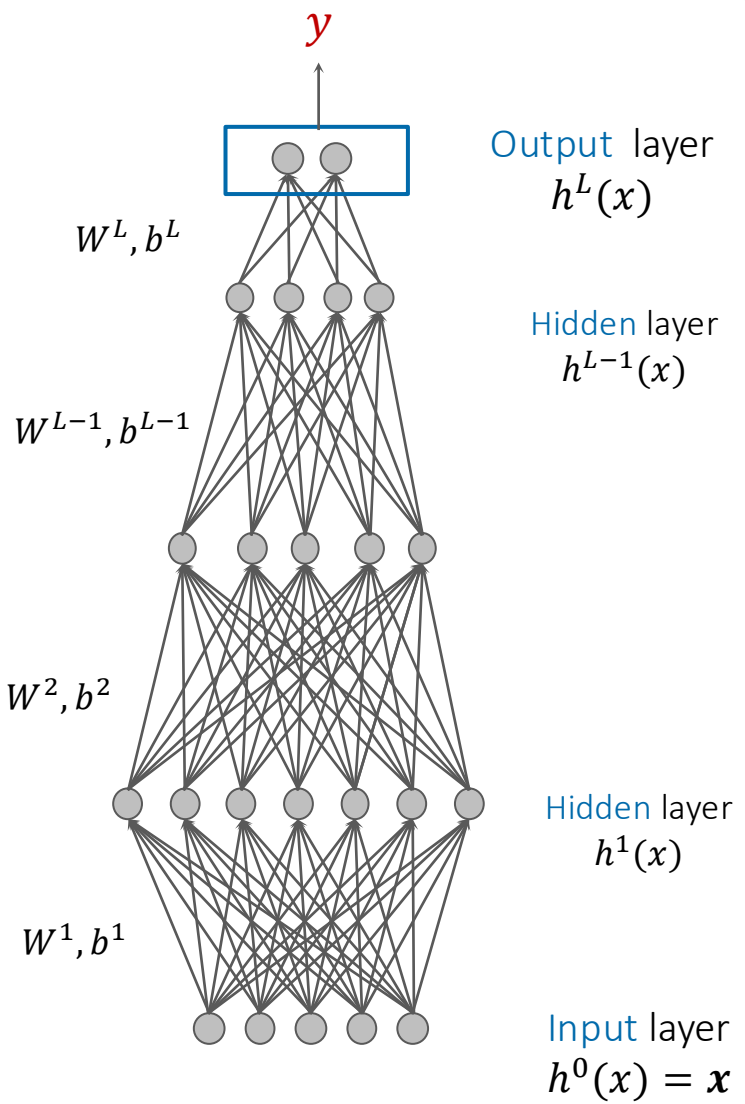


Recurrent NNs (RNNs)

DNN and CNNs





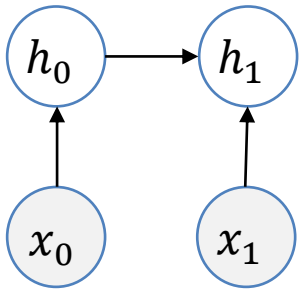


- DNNs: build networks for deep static structures
- CNNs: mainly builds network to exploit spatial patterns
- How to build network for sequences?

Recurrent Neural Networks

[Rumelhart, et al., 1986]

Simplest RNN with two time-slices (no output)

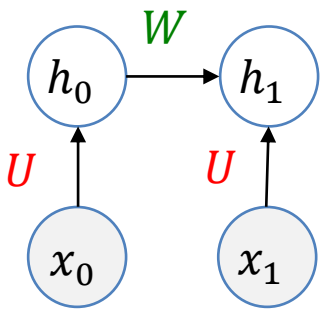


values known from data

hidden nodes, values to be inferred from our models

Recurrent Neural Networks

Simplest RNN with two time-slices (no output)



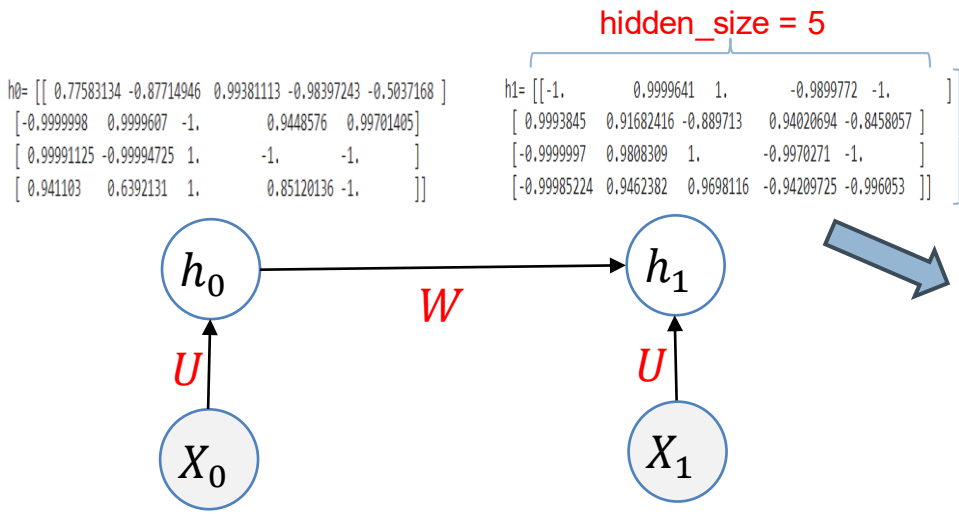
values known from data

hidden nodes, values to be inferred from our models

- Input: $x_0 \in \mathbb{R}^{1\times input_size}$, $x_1 \in \mathbb{R}^{1\times input_size}$
- $h_0 = \tanh(x_0 U + b) \in \mathbb{R}^{1\times hidden_size}$
  - $U \in \mathbb{R}^{input_size\times hidden_size}$
- $h_1 =$ some function of h_0 and x_1

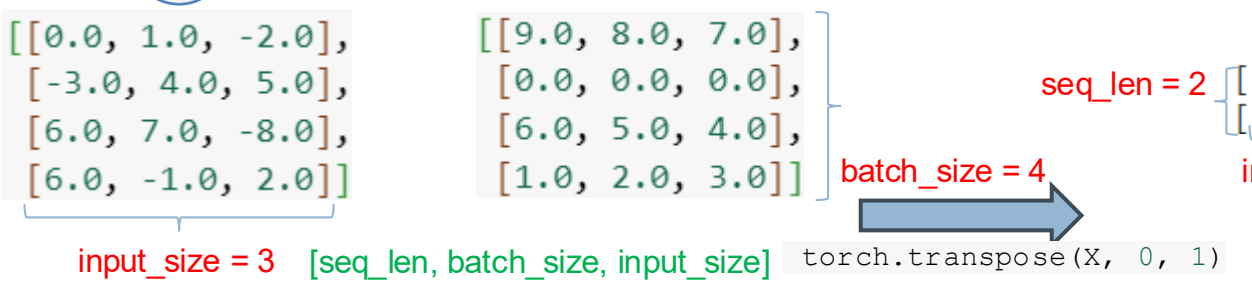
  $= \tanh(h_0 W + x_1 U + b) \in \mathbb{R}^{1\times hidden_size}$
  - $W \in \mathbb{R}^{hidden_size\times hidden_size}$

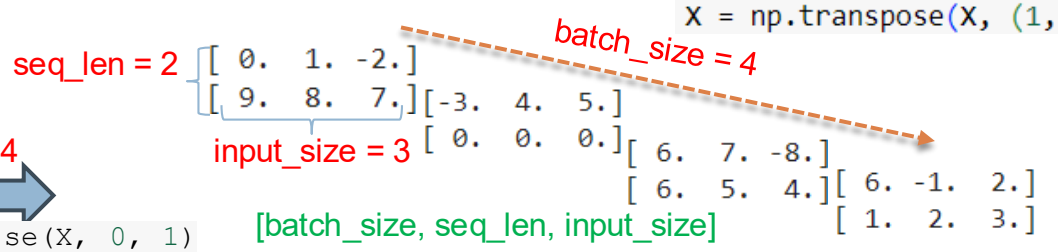
Recurrent Neural Networks



- Input: $X_0, X_1 \in \mathbb{R}^{bs \times input_size}$
- $h_0 = \tanh(X_0 U + b) \in \mathbb{R}^{bs \times hidden_size}$
  - $U \in \mathbb{R}^{input_size \times hidden_size}$
- $h_1 = \tanh(h_0 W + X_1 U + b) \in \mathbb{R}^{1 \times hidden_size}$

```
X= np.stack((X0, X1), axis=0)
X = np.transpose(X, (1, 0, 2))
```






```
import numpy as np

X0 = np.array([[0.0, 1.0, -2.0],
               [-3.0, 4.0, 5.0],
               [6.0, 7.0, -8.0],
               [6.0, -1.0, 2.0]], dtype= np.float32) # t = 0
X1 = np.array([[9.0, 8.0, 7.0],
               [0.0, 0.0, 0.0],
               [6.0, 5.0, 4.0],
               [1.0, 2.0, 3.0]], dtype= np.float32) # t = 1
```

```
hidden_size = 5
input_size = 3

# Creating the parameters
U = torch.nn.Parameter(torch.randn(input_size, hidden_size, dtype=torch.float32))
W = torch.nn.Parameter(torch.randn(hidden_size, hidden_size, dtype=torch.float32))
b = torch.nn.Parameter(torch.zeros(1, hidden_size, dtype=torch.float32))

X0 = torch.tensor(X0)
X1 = torch.tensor(X1)
# Implementing the operations
h0 = torch.tanh(torch.matmul(X0, U) + b)
h1 = torch.tanh(torch.matmul(X1, U) + torch.matmul(h0, W) + b)
```

```
print("h1= {}".format(h1.detach().numpy()))
```

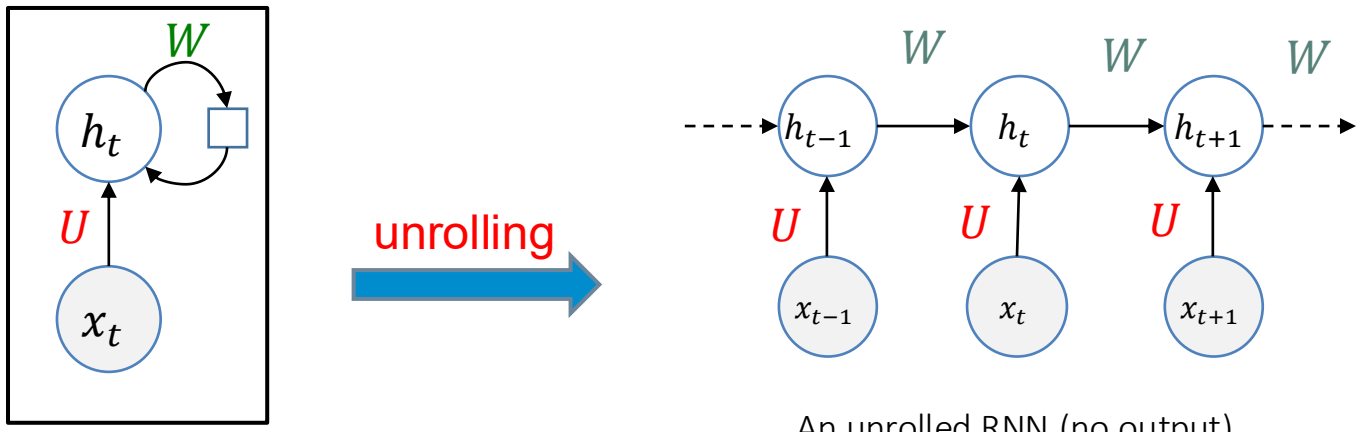
```
h1= [[-1.          0.9999641   1.         -0.9899772  -1.        ]
 [ 0.9993845   0.91682416 -0.889713   0.94020694 -0.8458057 ]
 [-0.9999997   0.9808309   1.         -0.9970271  -1.        ]
 [-0.99985224  0.9462382   0.9698116  -0.94209725 -0.996053  ]]
```

```
print("h0= {}".format(h0.detach().numpy()))
```

```
h0= [[ 0.77583134 -0.87714946  0.99381113 -0.98397243 -0.5037168 ]
 [-0.9999998   0.9999607  -1.          0.9448576   0.99701405]
 [ 0.99991125 -0.99994725  1.         -1.         -1.        ]
 [ 0.941103    0.6392131   1.          0.85120136 -1.        ]]
```

Recurrent Neural Networks

with no output



An unrolled RNN (no output)

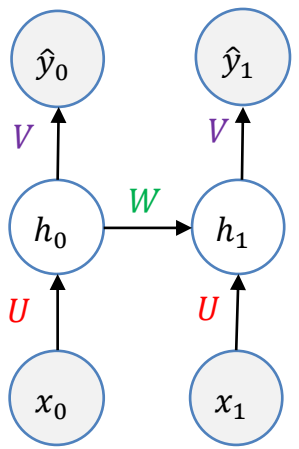
- **Idea**: *sharing parameters* for each data of the time index
- Given a data sequence $x_1, x_2, \dots, x_T$
- RNN models a dynamic system driven by an external signal x_t

$$h_t = f(h_{t-1}, x_t) = f(f(h_{t-2}, x_{t-1}), x_t) = \cdots = \text{summary}(x_{1:t}, h_0)$$

- h_t can be considered as a kind of lossy summary of the history $x_{1:t}$

Recurrent Neural Networks

Parameterization - 2 time slices

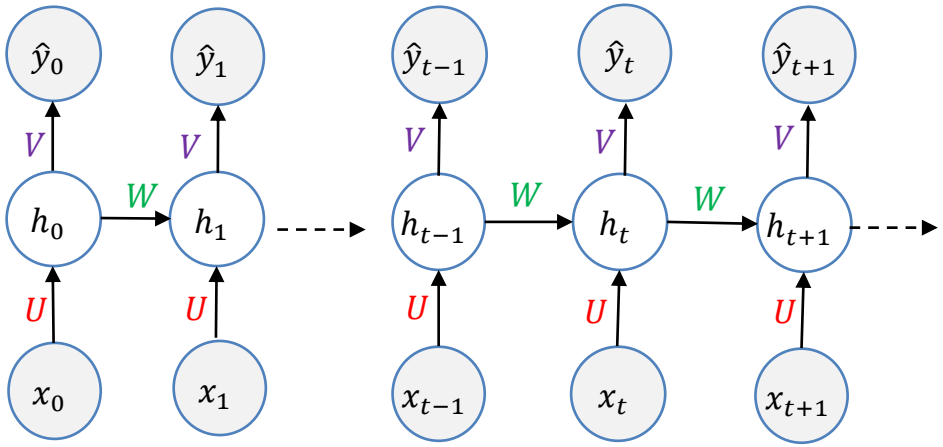


Simplest RNN with two time-slices (with output)

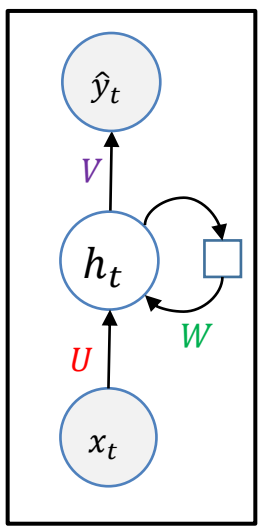
- Input: $x_0, x_1 \in \mathbb{R}^{1\times input_size}$, $y_0, y_1 \in Y$
- $h_0 = \tanh(x_0 U + b)$
- $\hat{y}_0 = \begin{cases} h_0 V + c \quad \text{(regression)} \\ \text{softmax}\,(h_0 V + c) \quad \text{(classification)} \end{cases}$
  - Suffer loss $l(\hat{y}_0, y_0)$
- $h_1 = \text{some function of } h_0 \text{ and } x_1$
  $= \tanh(h_0 W + x_1 U + b)$
- $\hat{y}_1 = \begin{cases} h_1 V + c \quad \text{(regression)} \\ \text{softmax}\,(h_1 V + c) \quad \text{(classification)} \end{cases}$
  - Suffer loss $l(\hat{y}_1, y_1)$
- $Total_loss = l(\hat{y}_0, y_0) + l(\hat{y}_1, y_1)$

Recurrent Neural Networks

Parametrization over multiple time slices



Simplest RNN with multiple time-slices (with output)



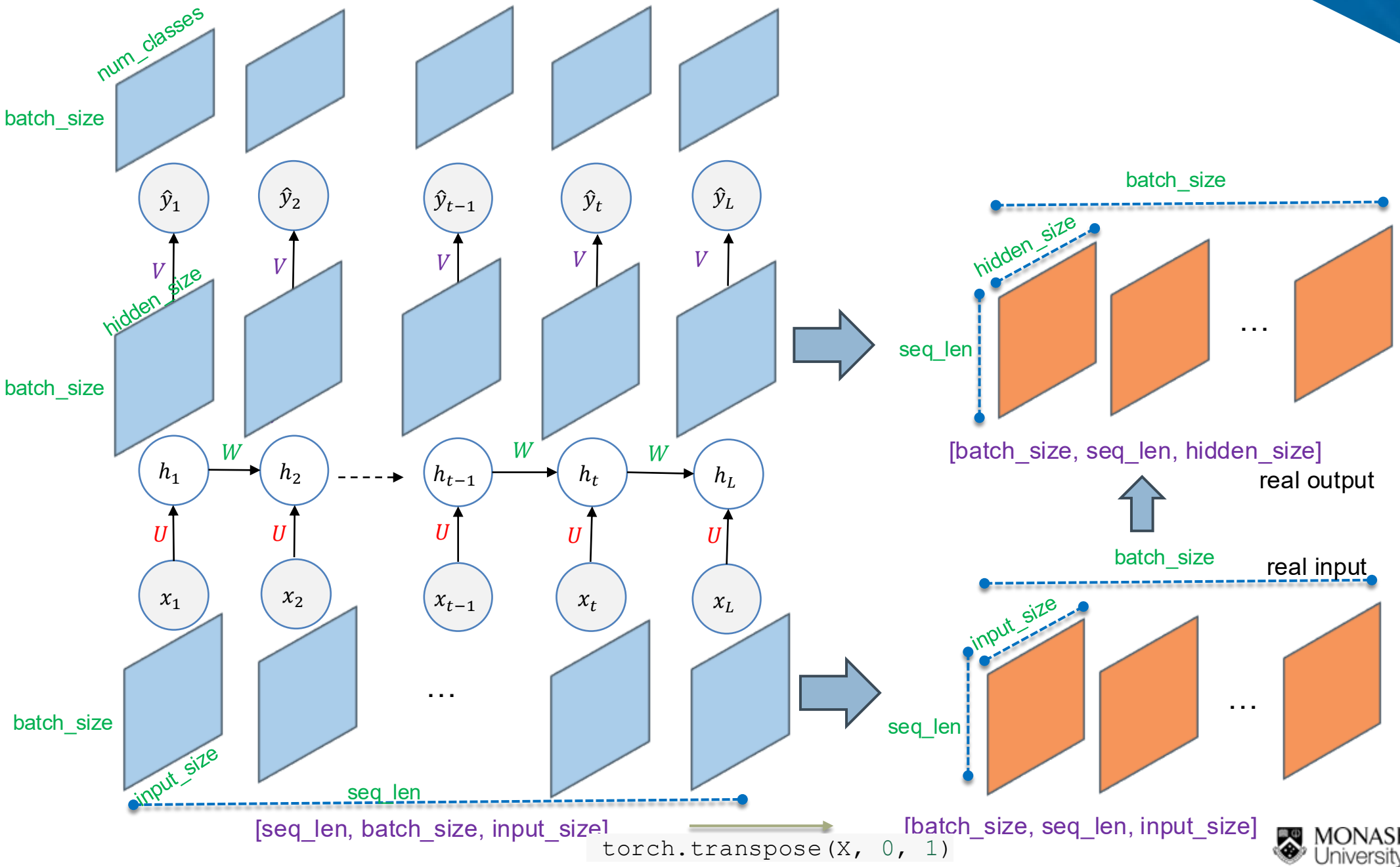
- Input: $x_0, x_1, .., x_t, \ldots \in \mathbb{R}^{1\times input_size}$, $y_0, y_1, \ldots \in Y$
- $h_0 = \tanh(x_0 U + b)$
- $\hat{y}_0 = \begin{cases} h_0 V + c & \text{(regression)} \\ \text{softmax}\,(h_0 V + c) & \text{(classification)} \end{cases}$
  - Suffer loss $l(\hat{y}_0, y_0)$
- for t=1,2,..., T

  h_t = some function of h_{t-1} and x_t

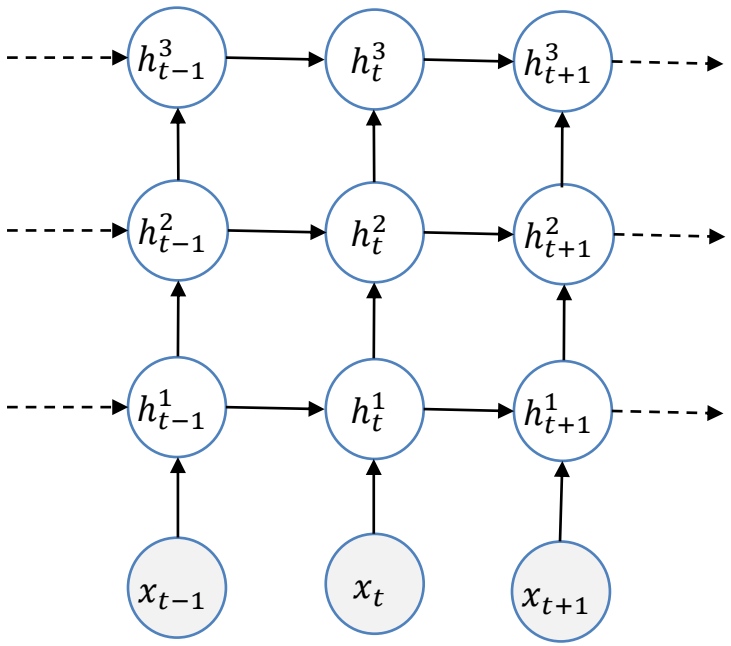
  $= \tanh(h_{t-1} W + x_t U + b)$

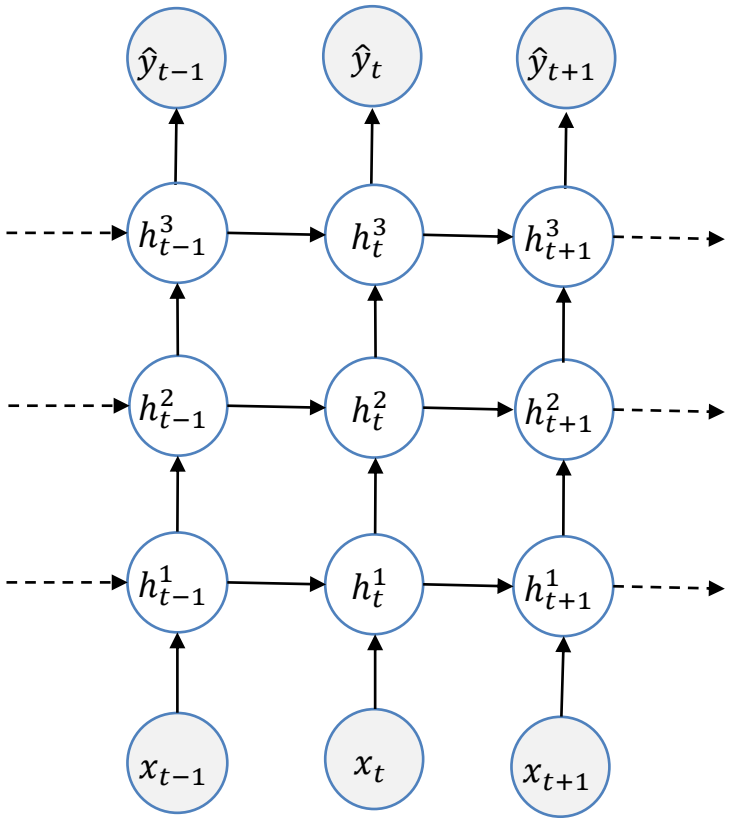
  $\hat{y}_t = \begin{cases} h_t V + c & \text{(regression)} \\ \text{softmax}\,(h_t V + c) & \text{(classification)} \end{cases}$
  - Suffer loss $l(\hat{y}_t, y_t)$
- $\boldsymbol{Total_loss} = \sum_{t=0}^{T} l(\hat{y}_t, y_t)$

Shape Transformation in RNNs



Deeper RNNs

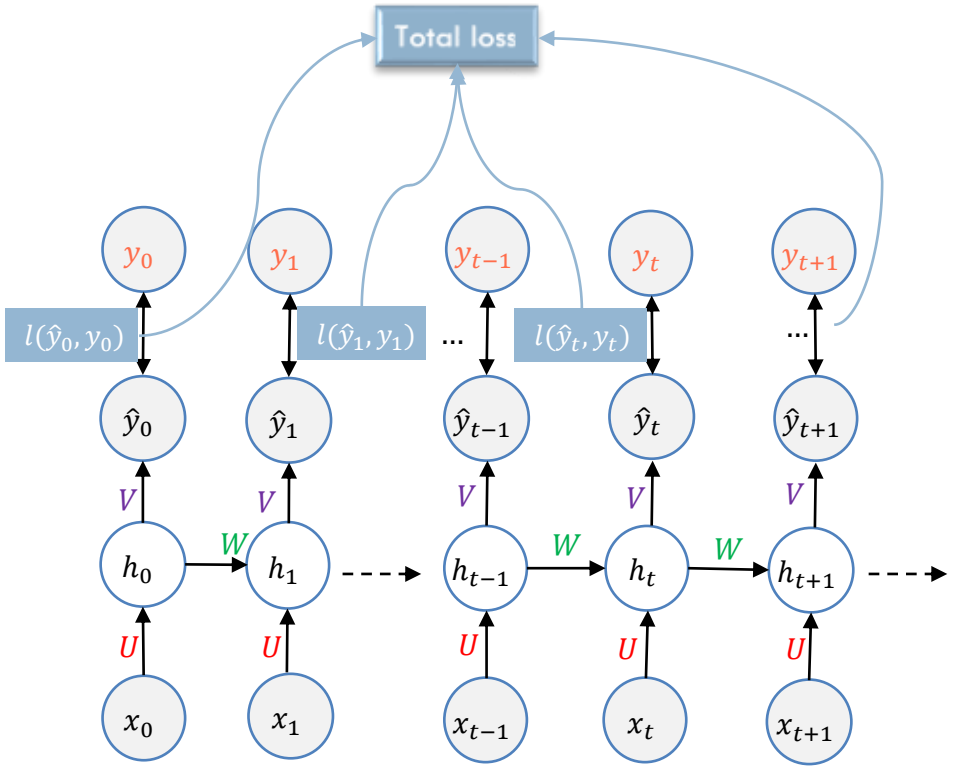

h_{t-1}^3
h_t^3
h_{t+1}^3
h_{t-1}^2
h_t^2
h_{t+1}^2
h_{t-1}^1
h_t^1
h_{t+1}^1
x_{t-1}
x_t
x_{t+1}


$\hat{y}_{t-1}$
$\hat{y}_t$
$\hat{y}_{t+1}$
h_{t-1}^3
h_t^3
h_{t+1}^3
h_{t-1}^2
h_t^2
h_{t+1}^2
h_{t-1}^1
h_t^1
h_{t+1}^1
x_{t-1}
x_t
x_{t+1}



Training RNNs with Back Propagation Through Time (BPTT)

Forward propagation through the time



- **Input:** $x_0, x_1, .., x_t, \ldots \in \mathbb{R}^{1\times input_size}$, $y_0, y_1, \ldots, y_t, \ldots \in Y$
- $\bar{h}_0 = x_0 U + b, h_0 = \tanh(\bar{h}_0)$
- $\hat{\boldsymbol{y}}_0 = \begin{cases} h_0 V + c \ \ (\text{regression}) \\ \text{softmax}\ (h_0 V + c) \ \ (\text{classification}) \end{cases}$
  - Suffer loss: $\text{loss}_0 = l(\hat{y}_0, y_0)$
- for t=1,2,...

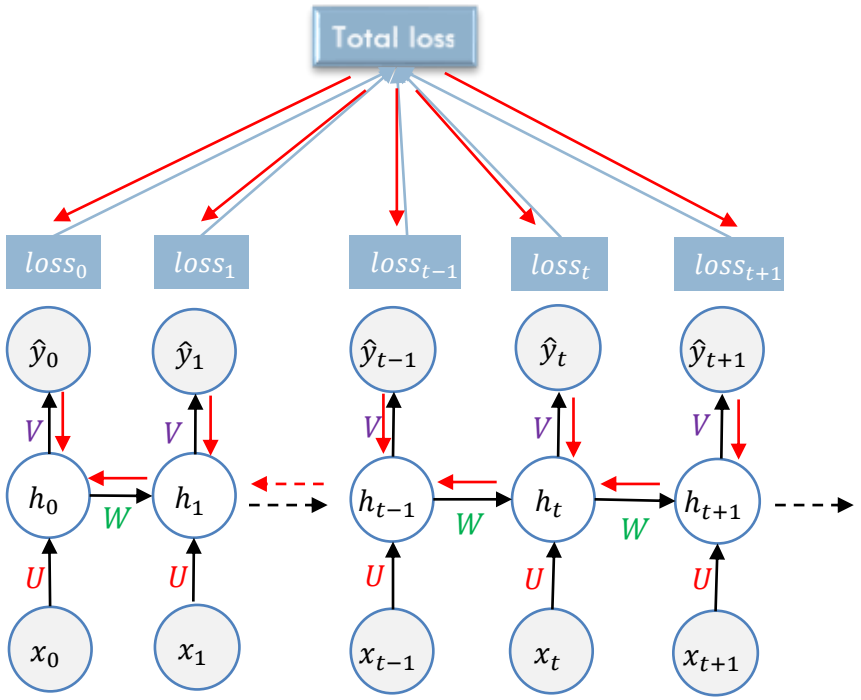
  Pre-activation: $\bar{h}_t = h_{t-1} W + x_t U + b$

  After-activation: $h_t = \tanh(\bar{h}_t)$

  Prediction: $\hat{\boldsymbol{y}}_t = \begin{cases} h_t V + c \ \ (\text{regression}) \\ \text{softmax}\ (h_t V + c) \ \ (\text{classification}) \end{cases}$

  Suffer loss: $\text{loss}_t = l(\hat{y}_t, y_t)$
- **Total loss**
  - $Total - loss = \sum_t loss_t$

Back propagation through the time



Pre-activation: $\bar{h}_t = h_{t-1}W + x_tU + b$

After-activation: $h_t = \tanh(\bar{h}_t)$

Prediction: $\hat{y}_t = \begin{cases} h_tV + c & \text{(regression)} \\ \text{softmax}\,(h_tV + c) & \text{(classification)} \end{cases}$

Suffer loss $loss_t = \ell(\hat{y}_t, y_t)$

- Total loss
  - $L = \sum_t loss_t$
- Let us compute $\frac{\partial L}{\partial h_0}$?
- $$\frac{\partial L}{\partial h_0} = \sum_t \frac{\partial loss_t}{\partial h_0}$$
$$= \sum_t \frac{\partial loss_t}{\partial \hat{y}_t}\frac{\partial \hat{y}_t}{\partial h_t}\frac{\partial h_t}{\partial \bar{h}_t}\frac{\partial \bar{h}_t}{\partial h_{t-1}} \cdots \frac{\partial h_1}{\partial \bar{h}_1}\frac{\partial \bar{h}_1}{\partial h_0}$$

$$\frac{\partial L}{\partial h_0} = \sum_t \frac{\partial loss_t}{\partial \hat{y}_t}\frac{\partial \hat{y}_t}{\partial h_t} diag\left(1-\bar{h}_t^2\right)W \ldots diag\left(1-\bar{h}_1^2\right)W$$

Multiplication of many W

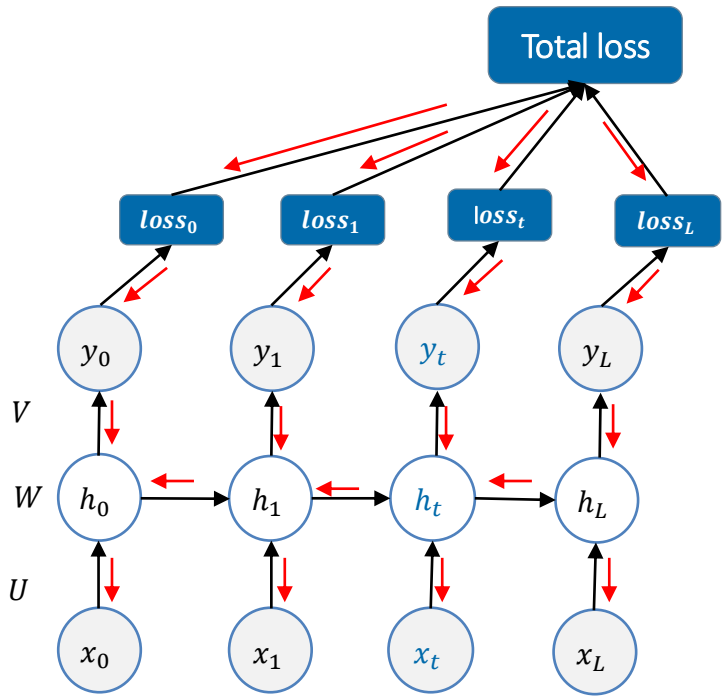
❖ Gradient vanishing/exploding

Not in assessment

Training RNN

Summary

- Use BackProp Through Time (BPTT)
  - Create unrolled network
  - Forward pass, store values and calculate loss at each time slide
  - Backward propagation through time, starting from the last step, to calculate the local gradients at each time step.
  - Sum up local gradients to obtain the end gradients for each parameters
  - Update the parameters via GD/SGD



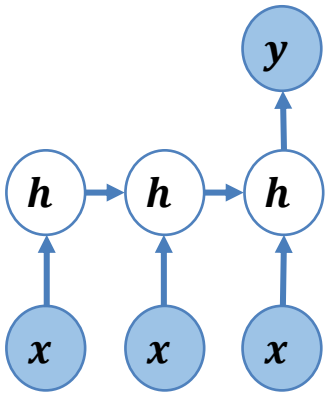
Not in assessment



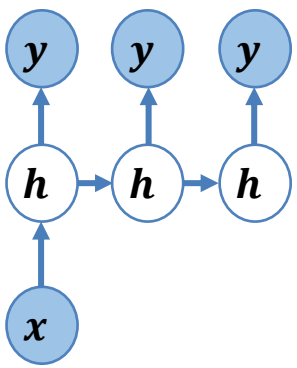
RNNs applications

RNNs Architecture Zoo

- RNN's architecture is very flexible for many real-world tasks

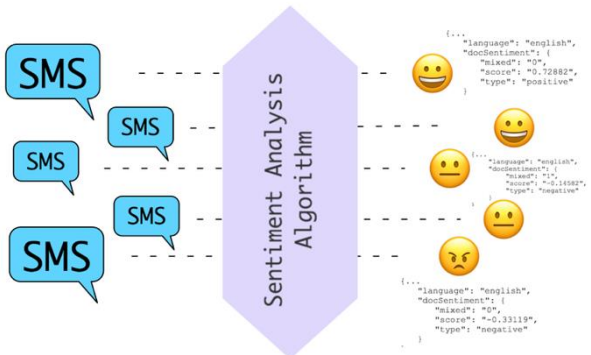


many to one
(sentiment analysis, image classification)



one to many
(image captioning)





Sentiment analysis
[https://www.twilio.com]

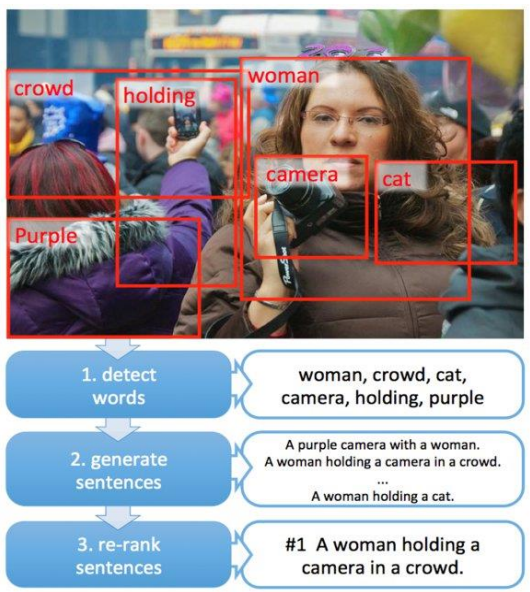
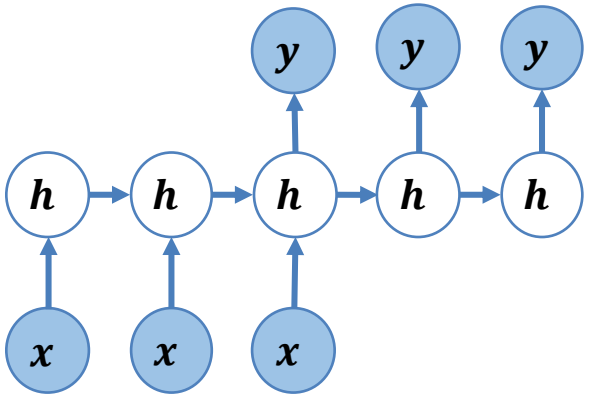


Image captioning
[https://www.pcworld.com]

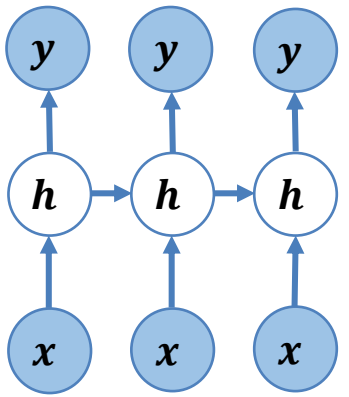
RNNs Architecture Zoo

- RNNs architecture is very flexible for many real-world tasks



many to many (1)

(machine translation)



many to many (2)

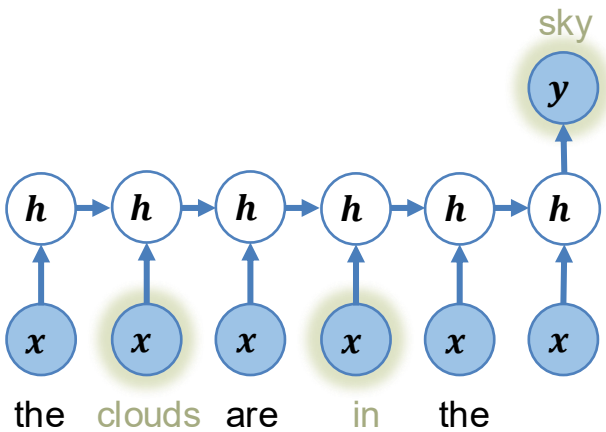
(video classification)

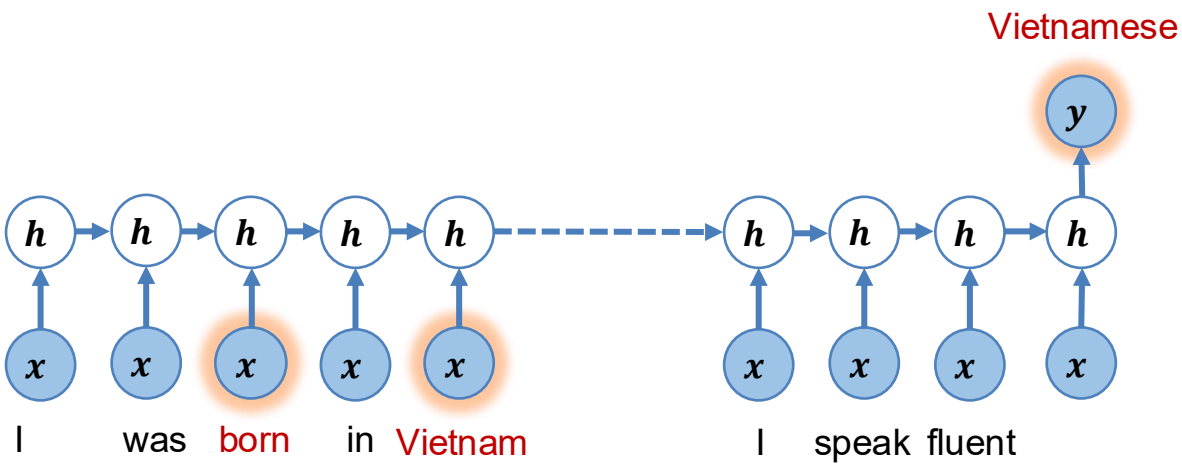
… more to come in our next lectures


Q&A
When?
Why?
How?
WHAT?
WHERE?
WHEN?
HOW?
WHO?
What?
Where?

Problems of simple RNN and rethinking the memory cell

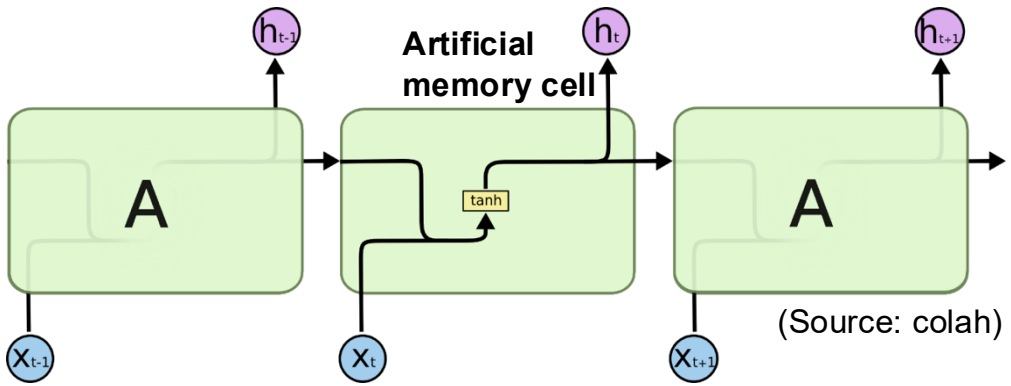
Problems of RNN

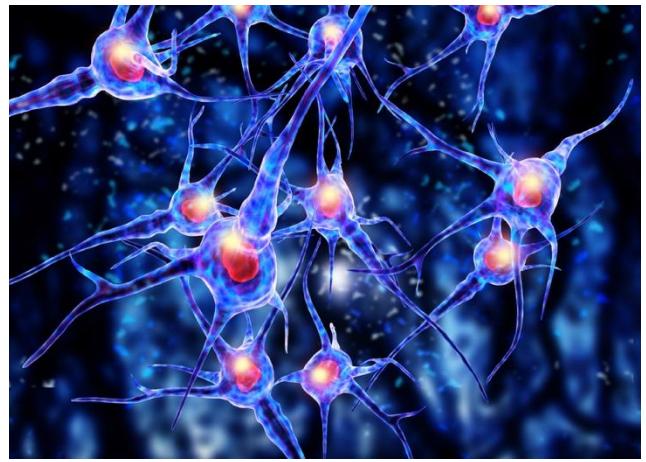




- RNNs **don't** capture **long-term dependency** adequately
  - A hidden state is computed based on only one previous state → can only capture **short-term dependency**
- Modelling drawbacks
  - Technical problem when training long sequences
    - vanishing gradient problem
  - Many layers of nonlinear transformation prevent the data signals and gradient from flowing easily through the network.
- How to address this?
  - Using gating mechanism: adding linear component from previous layer!
    - RNN → LSTM/GRU

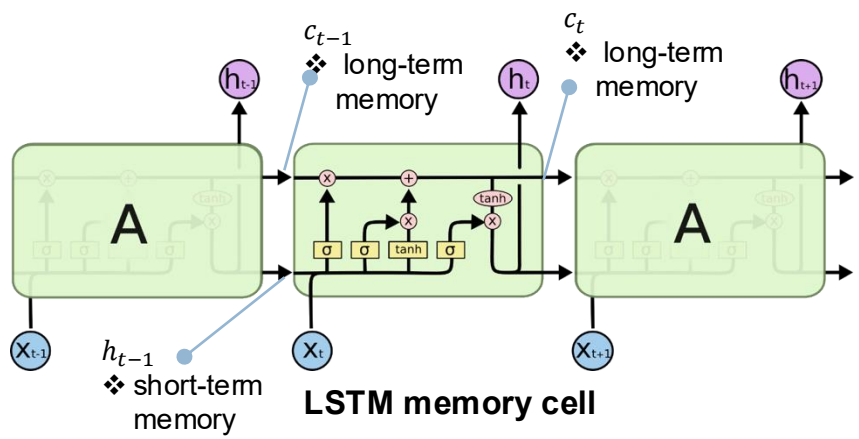
Memory cells





Biological **memory cells** in human brain
(Source: news.feinberg.northwestern.edu)

- Our RNN includes many **simple RNN cells**
  - **Input to a cell**: h_{t-1} (previous hidden state) and x_t (current input token)
  - **Output**: $h_t = \tanh(x_t U + h_{t-1} W + b)$
  - h_t can only capture **short-term dependency** → **short-term memory**
- How to capture **long-term memory** more efficiently?
  - **LSTM** cell and **GRU** cell

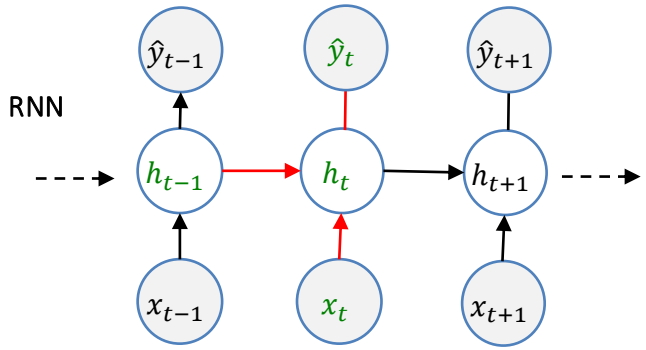


Long-Short Term Memory Cells (LSTM)

[Hochreiter and Schimdhuber ‘97]

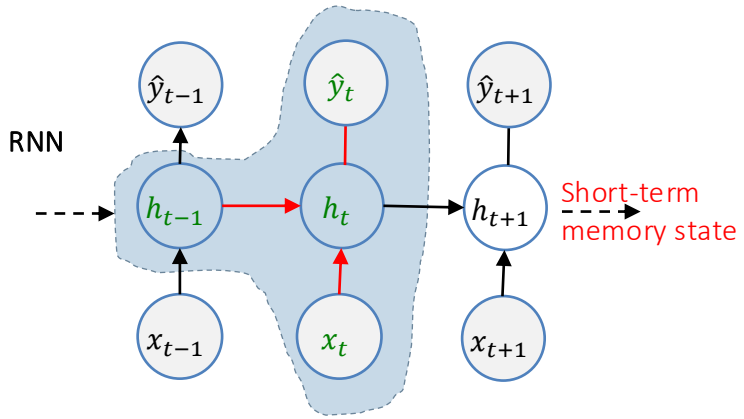
Long Short-Term Memory (LSTM)

[Hochreiter and Schimdhuber '97]



Long Short-Term Memory (LSTM)

[Hochreiter and Schimdhuber '97]

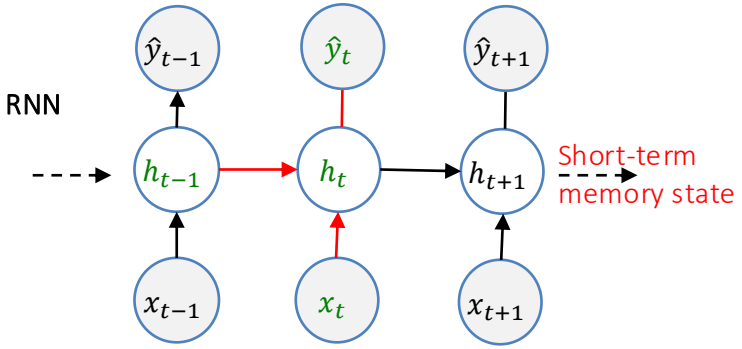


$$h_t = f(h_{t-1}, x_t) = f(f(h_{t-2}, x_{t-1}), x_t) = \cdots = \text{summary}(x_{1:t}, h_0)$$

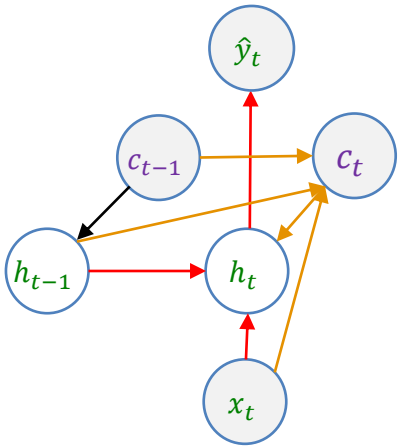
- $h_t = \tanh(h_{t-1}W + x_tU + b)$
- $\hat{y}_t = \text{softmax}(h_tV + c)$

Long Short-Term Memory (LSTM)

[Hochreiter and Schmiddhuber '97]



LSTM

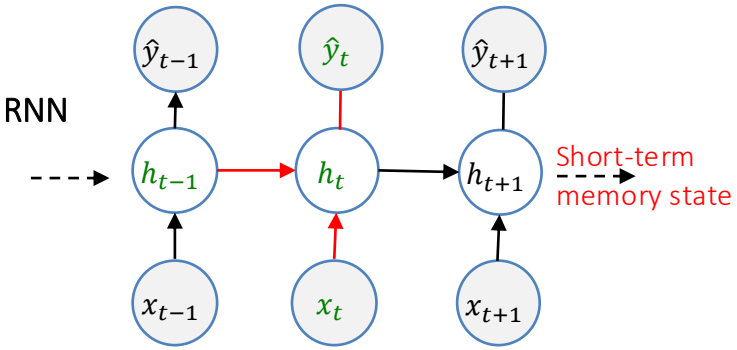


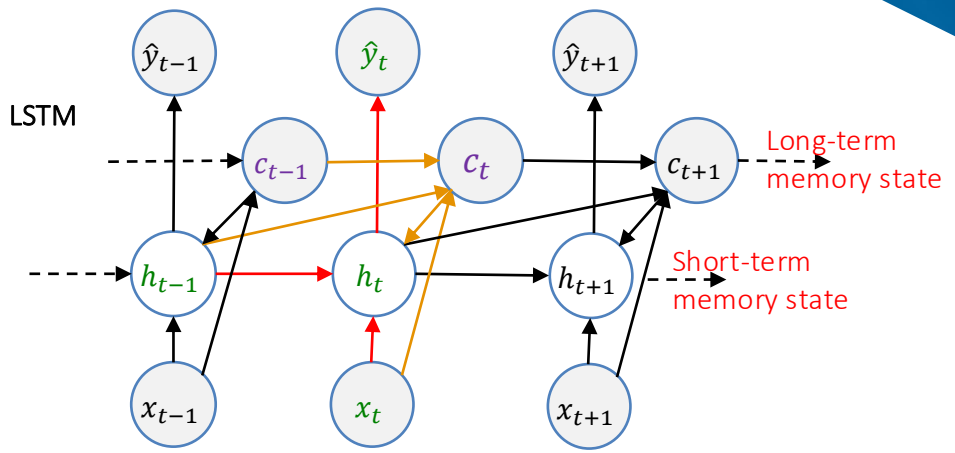
LSTM

- Introduced in **1997** by **Hohreiter and Schmidhuber** and improved over the years: Sak et. al 2014, Zaremba, 2015, etc.
- Address the **long-term dependency problem** by introducing a **long-term state memory** c_t
- Help the **gradient flow significantly** over a long duration, hence capture long-term dependency

Long Short-Term Memory (LSTM)

[Hochreiter and Schmiddhuber '97]



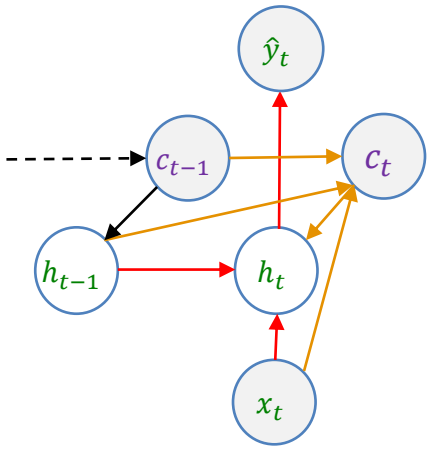


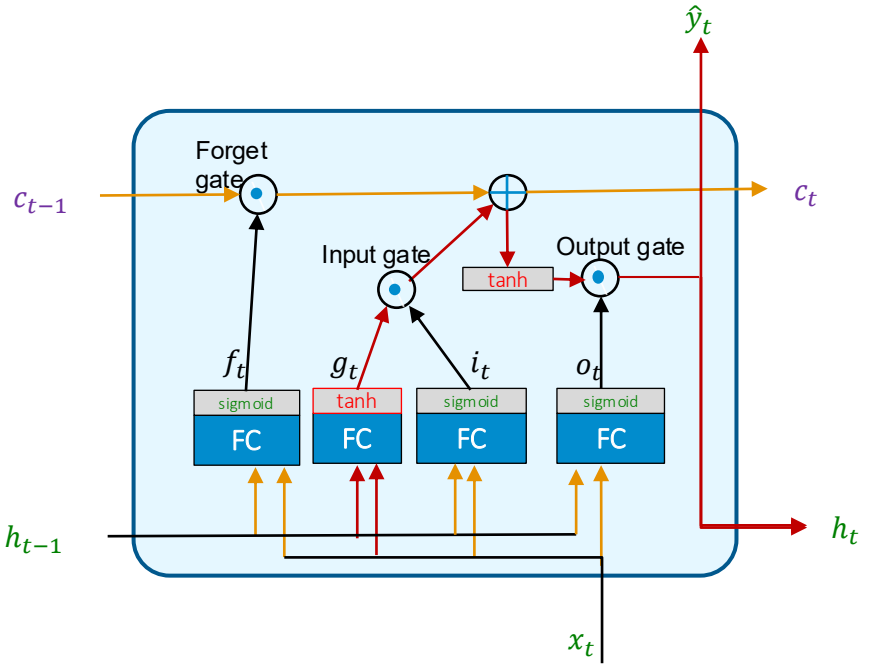
LSTM

- Introduced in **1997** by **Hohreiter and Schmidhuber** and improved over the years: Sak et. al 2014, Zaremba, 2015, etc.
- Address the **long-term dependency problem** by introducing a **long-term state memory** c_t
- Help the **gradient flow significantly** over a long duration, hence capture long-term dependency

Long Short-Term Memory (LSTM)

Output Gate



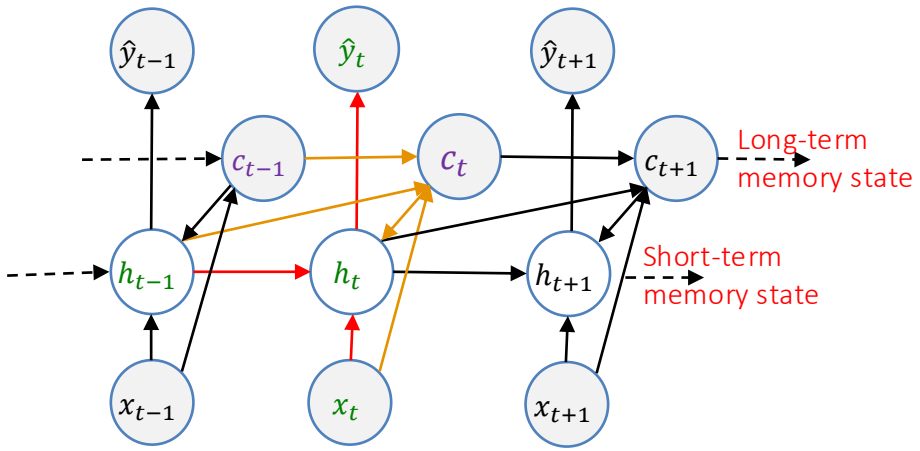


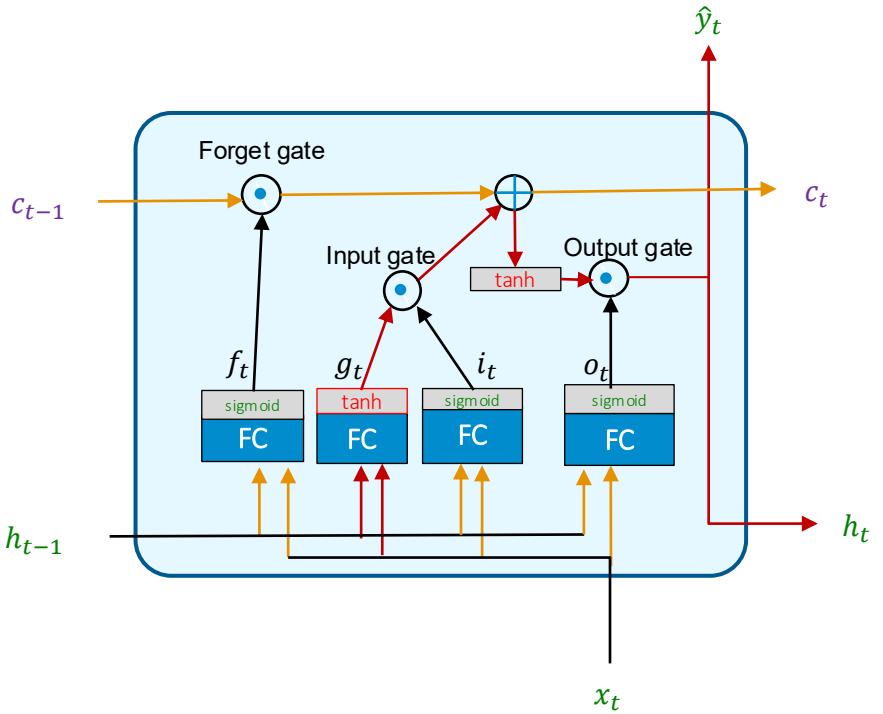
- Introduce three gate controllers:
  - Use sigmoid to ensure outputs' range between 0 and 1
  - Forget gate f_t controls how much c_{t-1} be 'forgotten'
  - Input gate i_t controls which how much g_t be remembered
  - Output gate o_t controls how much long-term c_t should be carried on to the next time slice:
    - to contribute to short-term state: h_t

- $g_t = \tanh(h_{t-1}W + x_tU + b)$
- Forget gate: $f_t = \sigma(x_tU^f + h_{t-1}W^f + b^f)$
- Input gate: $i_t = \sigma(x_tU^i + h_{t-1}W^i + b^i)$
- LSTM long-term state: $c_t = f_t \odot c_{t-1} + g_t \odot i_t$
- Output gate: $o_t = \sigma(x_tU^o + h_{t-1}W^o + b^o)$
- LSTM short-term state: $h_t = o_t \odot \tanh(c_t)$

Long Short-Term Memory (LSTM)

Output Gate





- Regression

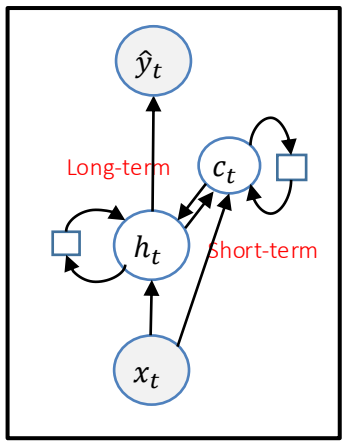
$$\hat{y}_t = \boldsymbol{h_t V} + \boldsymbol{c}$$

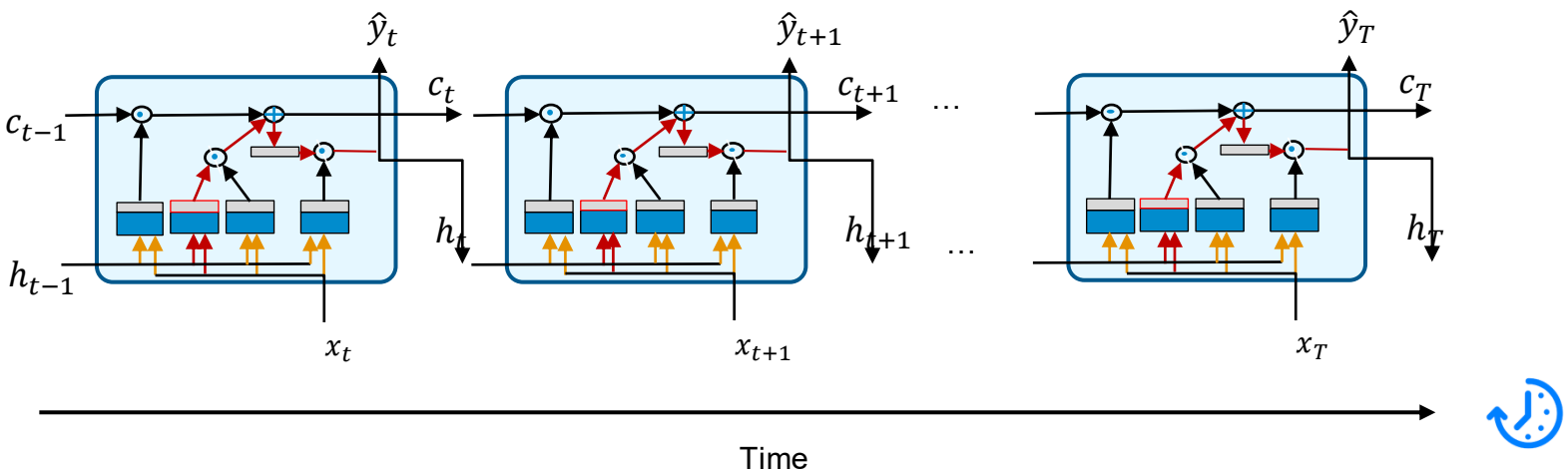
- Classification

$$\hat{y}_t = \text{softmax}(\boldsymbol{h_t V} + \boldsymbol{c})$$

- $g_t = \tanh(h_{t-1}W + x_t U + b)$
- Forget gate: $f_t = \sigma\left(x_t U^f + h_{t-1} W^f + b^f\right)$
- Input gate: $i_t = \sigma(x_t U^i + h_{t-1} W^i + b^i)$
- LSTM long-term state: $c_t = f_t \odot c_{t-1} + g_t \odot i_t$
- Output gate: $o_t = \sigma(x_t U^o + h_{t-1} W^o + b^o)$
- LSTM short-term state: $h_t = o_t \odot \tanh(c_t)$
- LSTM output: $\hat{\boldsymbol{y}}_{\boldsymbol{t}} = \boldsymbol{h_t V} + \boldsymbol{c}$ or $\hat{\boldsymbol{y}}_{\boldsymbol{t}} = \text{softmax}(\boldsymbol{h_t V} + \boldsymbol{c})$

Long Short-Term Memory (LSTM)

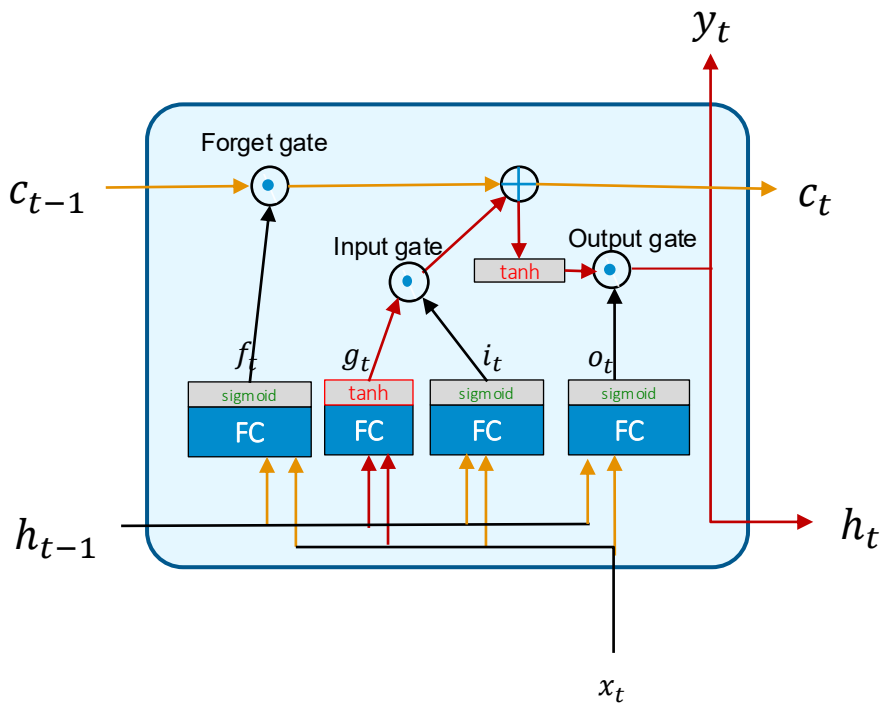




Long Short-Term Memory (LSTM)

Summary

- LSTM belongs to a class of gated RNN models
- LSTM introduces self-loops to create paths where the gradient can flow for long durations
- Improve over basic RNN cell
  - Can capture long-term dependency
  - Faster and more robust to train, often with quicker convergence
- LSTM cells manage two state vectors, and for performance reasons they are kept separate by default
  - h_t as the short-term state
  - c_t as the long-term state
- Gates can remove or add information to the cell state: forget, input, output



Gated Recurrent Unit (GRU)

[Cho et. al, 2014]

Gated Recurrent Unit

[Cho et. al, 2014]

- Proposed by Cho et al. 2014
  - Was introduced for Encoder-Decoder network
- Can be viewed as a **simplified version** of the LSTM:
  - Both long-term c_t and short-term h_t are **merged** to a single state h_t
  - Single **update gate** controller z_t is used to control both forget and input gates
    - = 1: open input gate, close forget gate
    - = 0: close input gate, open forget gate
    - i.e., whenever a memory must be stored, the location to be stored will be erased first!
  - Output gate is removed, but an additional reset gate r_t controls how much previous state should be carried forward

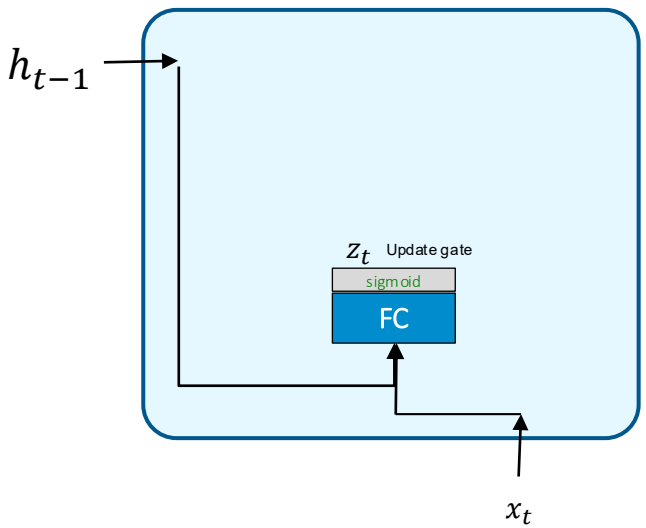
Cho et al., Learning Phrase Representation using RNN Encoder-Decoder for Statistical Machine Translation, *Proceedings of EMNLP*, 2014.

Gated Recurrent Unit

[Cho et. al, 2014]

- Update gate z_t decides how much the unit updates its state:

$$z_t = \sigma(x_t U^z + h_{t-1} W^z)$$



Gated Recurrent Unit

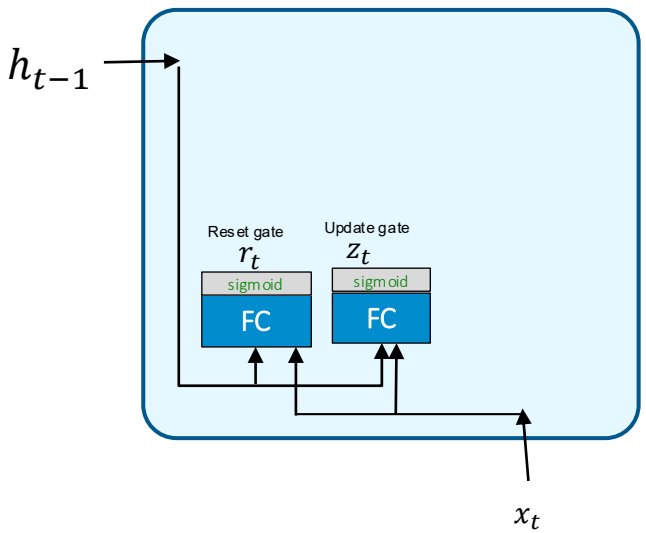
[Cho et. al, 2014]

- Update gate z_t decides how much the unit updates its state:

$$z_t = \sigma(x_t U^z + h_{t-1} W^z)$$

- Reset gate controls which parts of the state get used to compute the next target state

$$r_t = \sigma(x_t U^r + h_{t-1} W^r)$$



Gated Recurrent Unit

[Cho et. al, 2014]

- Update gate z_t decides how much the unit updates its state:

$$z_t = \sigma(x_t U^z + h_{t-1} W^z)$$

- Reset gate controls which parts of the state get used to compute the next target state

$$r_t = \sigma(x_t U^r + h_{t-1} W^r)$$

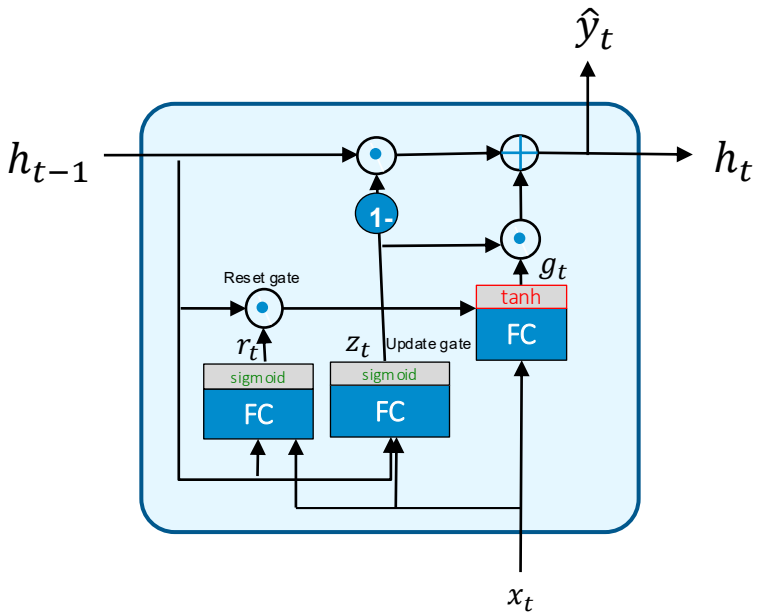
- The memory state h_t is a linear interpolation between h_{t-1} and g_t

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot g_t$$

where the candidate g_t is pre-computed

$$g_t = \tanh(x_t U^g + (r_t \odot h_{t-1}) W^g)$$

- When z_t and r_t are close to 1, GRU will be reduced to Basic RNN

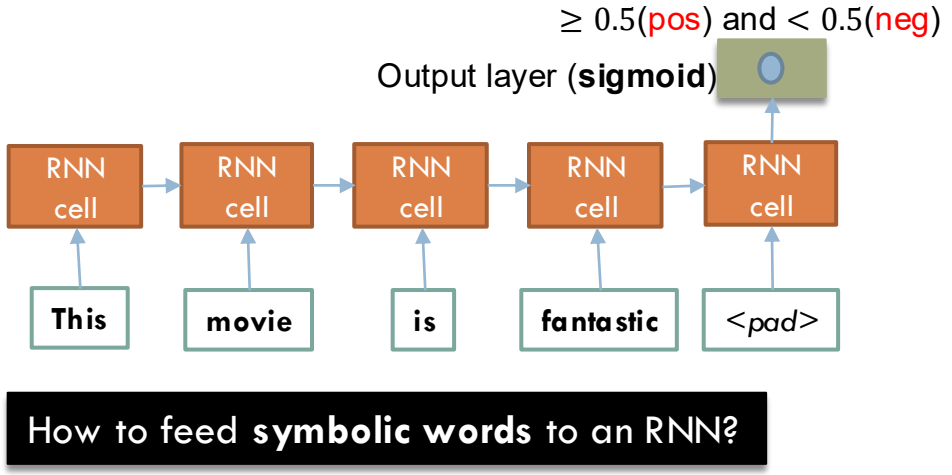


Applications of Recurrent Neural Networks

- Sentiment Analysis
- Text generation

Sentiment analysis

- **Movie review dataset**
  1. I like that movie (**pos**:1).
  2. This is a bad movie to watch (**neg**:0)
  3. I love the movie (**pos**: 1)
  4. I do not recommend you to watch this movie (**neg**:0)
  5. This movie is fantastic (**pos**:1)

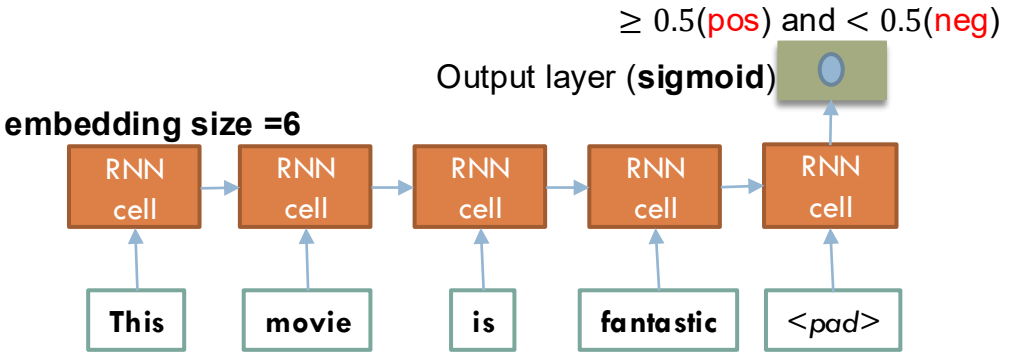


Sentiment analysis

- **Movie review dataset**
  1. I like this movie (**pos**:1).
  2. This is a bad movie to watch (**neg**:0)
  3. I love this movie (**pos**: 1)
  4. I do not recommend you to watch this movie (**neg**:0)
  5. This movie is fantastic (**pos**:1)

- **Build up vocabulary**
  1. Like (index: 1)
  2. Love (index: 2)
  3. Bad (index: 3)
  4. Fantastic (index: 4)
  5. Not (index: 5)
  6. Recommend (index: 6)

- **Not in vocabulary** (out of vocabulary bucket: 2)
  1. I, movie, to, pad (index: 7)
  2. This, is, watch (index: 8)



Embedding matrix (E [8×6])

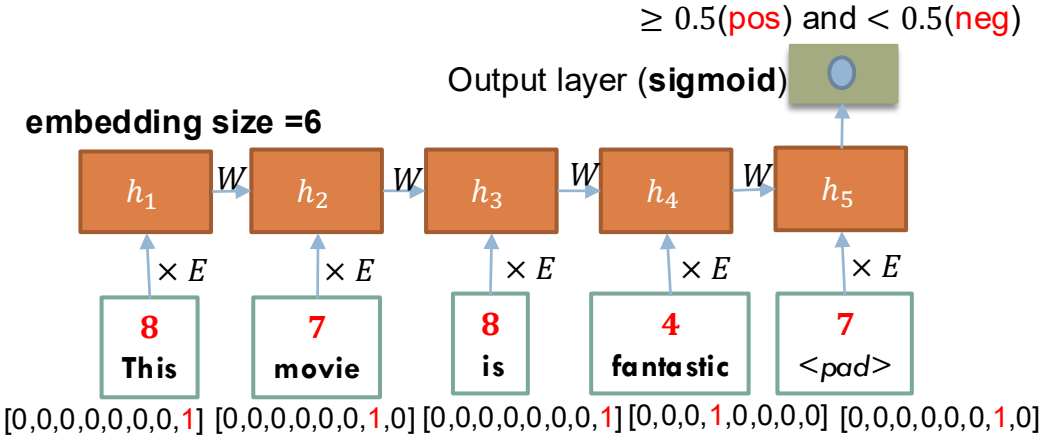
E_1	1	2	1.5	-1.2	1.3	1
E_2	-1	1.3	-2.5	-1.2	1.6	-1
E_3	1	3.3	-3.5	-1.0	2.6	1
E_4	-1	1.3	-2.5	-1.2	1.8	-1
E_5	-1.2	2.3	-2.5	-1.2	-1.6	1
E_6	-1.7	-1.3	-2.5	-1.2	3.6	1.2
E_7	-4.2	2.3	-3.5	4.3	1.8	-2
E_8	-1.7	-1.3	-4.5	-2.2	-3.6	1

Sentiment analysis

- **Movie review dataset**
  1. I like this movie (**pos**:1).
  2. This is a bad movie to watch (**neg**:0)
  3. I love this movie (**pos**: 1)
  4. I do not recommend you to watch this movie (**neg**:0)
  5. This movie is fantastic (**pos**:1)

- **Build up vocabulary**
  1. Like (index: 1)
  2. Love (index: 2)
  3. Bad (index: 3)
  4. Fantastic (index: 4)
  5. Not (index: 5)
  6. Recommend (index: 6)

- **Not in vocabulary** (out of vocabulary bucket: 2)
  1. I, movie, to, pad (index: 7)
  2. This, is, watch (index: 8)

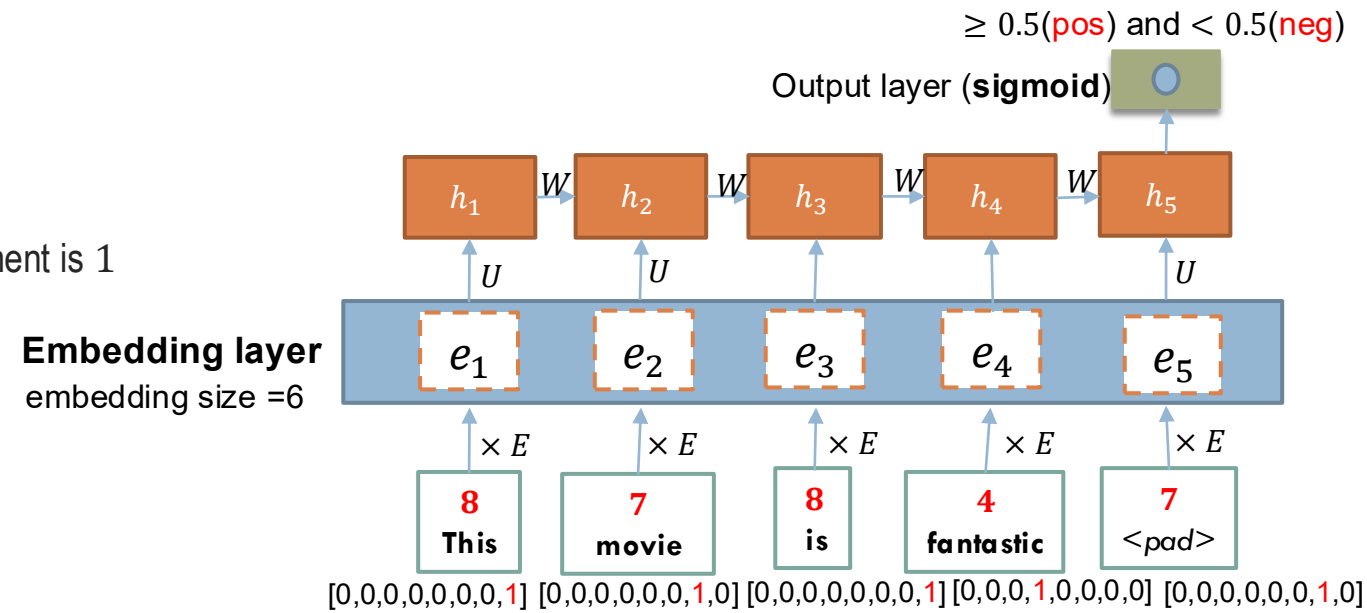


Embedding matrix (E [8×6])

E_1	1	2	1.5	-1.2	1.3	1
E_2	-1	1.3	-2.5	-1.2	1.6	-1
E_3	1	3.3	-3.5	-1.0	2.6	1
E_4	-1	1.3	-2.5	-1.2	1.8	-1
E_5	-1.2	2.3	-2.5	-1.2	-1.6	1
E_6	-1.7	-1.3	-2.5	-1.2	3.6	1.2
E_7	-4.2	2.3	-3.5	4.3	1.8	-2
E_8	-1.7	-1.3	-4.5	-2.2	-3.6	1

Sentiment analysis

- The word/item embedding
  - $e_1 = 1_8 E = E_8 \in \mathbb{R}^{1\times 6}$
    - $[1 \times 8] \times [8 \times 6] = [1 \times 6]$
    - 1_i is one-hot vector in which the $i-th$ element is 1 and the rest are 0.
  - $e_2 = 1_7 E = E_7 \in \mathbb{R}^{1\times 6}$
  - $e_3 = 1_8 E = E_8 \in \mathbb{R}^{1\times 6}$
  - $e_4 = 1_4 E = E_4 \in \mathbb{R}^{1\times 6}$
  - $e_5 = 1_7 E = E_7 \in \mathbb{R}^{1\times 6}$



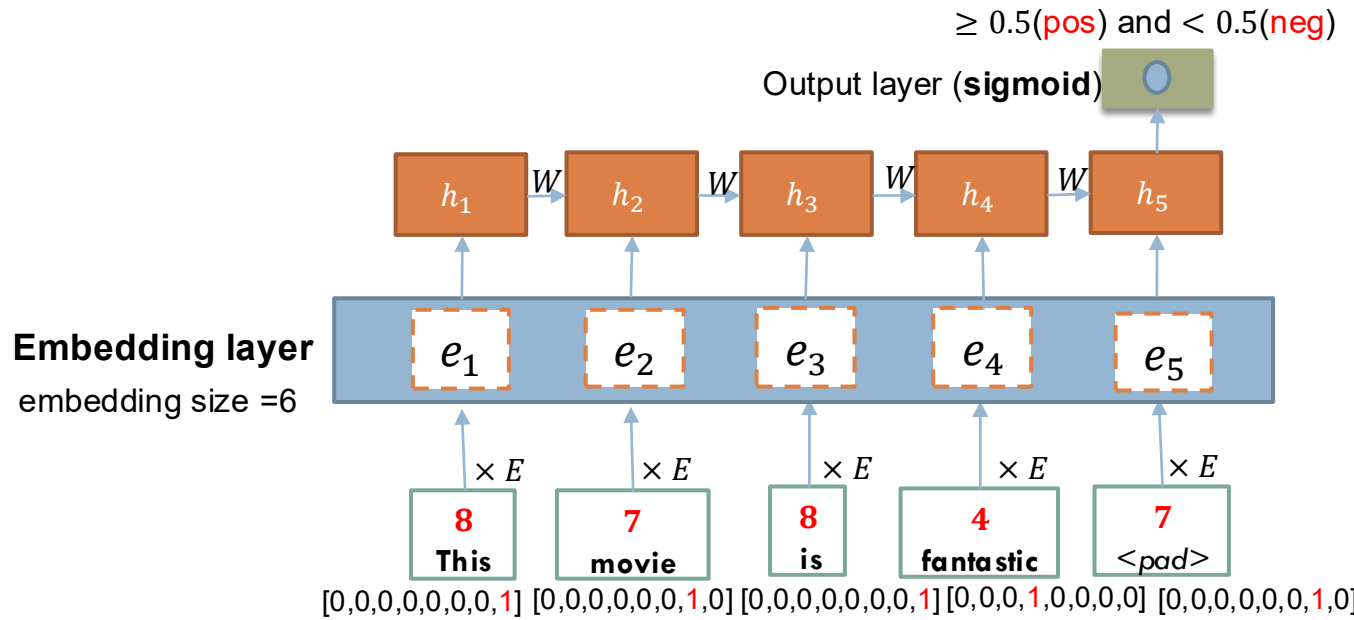
- The sequence embedding
  - $e = \begin{bmatrix} e_1 \\ e_2 \\ e_3 \\ e_4 \\ e_5 \end{bmatrix} \in \mathbb{R}^{5\times 6} = \mathbb{R}^{seq_len \times embed_size}$
  - **Embedding lookup** operation
    - Pick rows with indices **8, 7, 8, 4, 7**

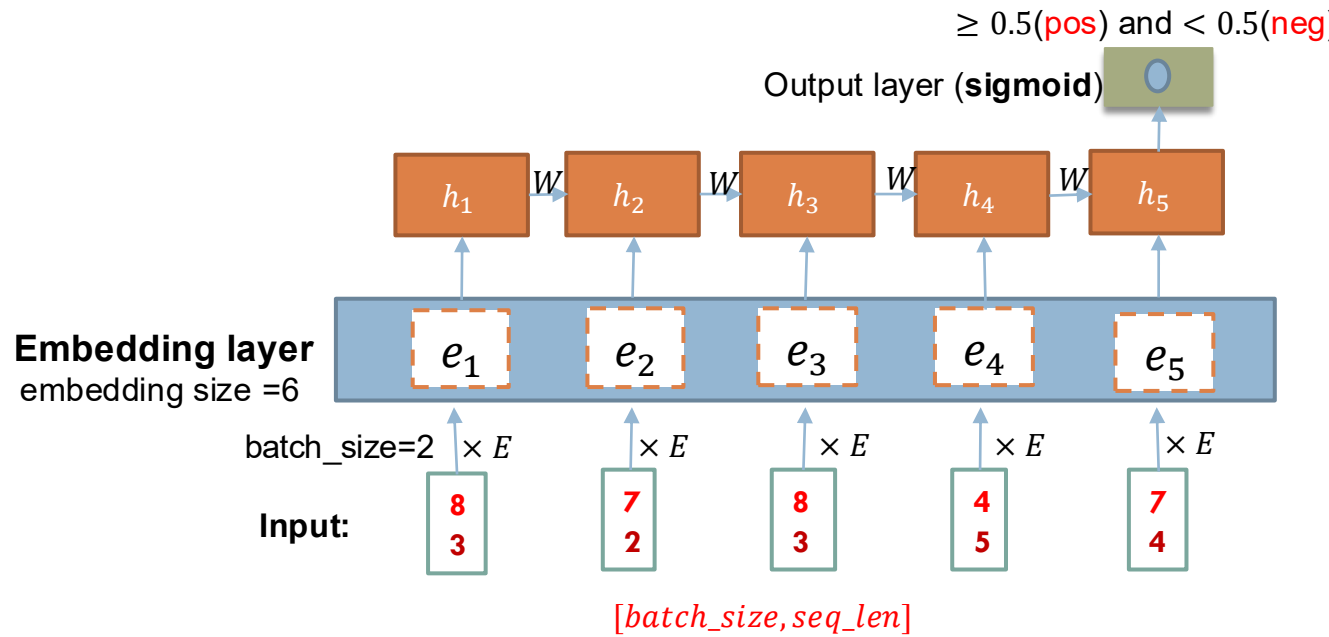
Embedding matrix (E [8×6])

E_1	1	2	1.5	-1.2	1.3	1
E_2	-1	1.3	-2.5	-1.2	1.6	-1
E_3	1	3.3	-3.5	-1.0	2.6	1
E_4	-1	1.3	-2.5	-1.2	1.8	-1
E_5	-1.2	2.3	-2.5	-1.2	-1.6	1
E_6	-1.7	-1.3	-2.5	-1.2	3.6	1.2
E_7	-4.2	2.3	-3.5	4.3	1.8	-2
E_8	-1.7	-1.3	-4.5	-2.2	-3.6	1

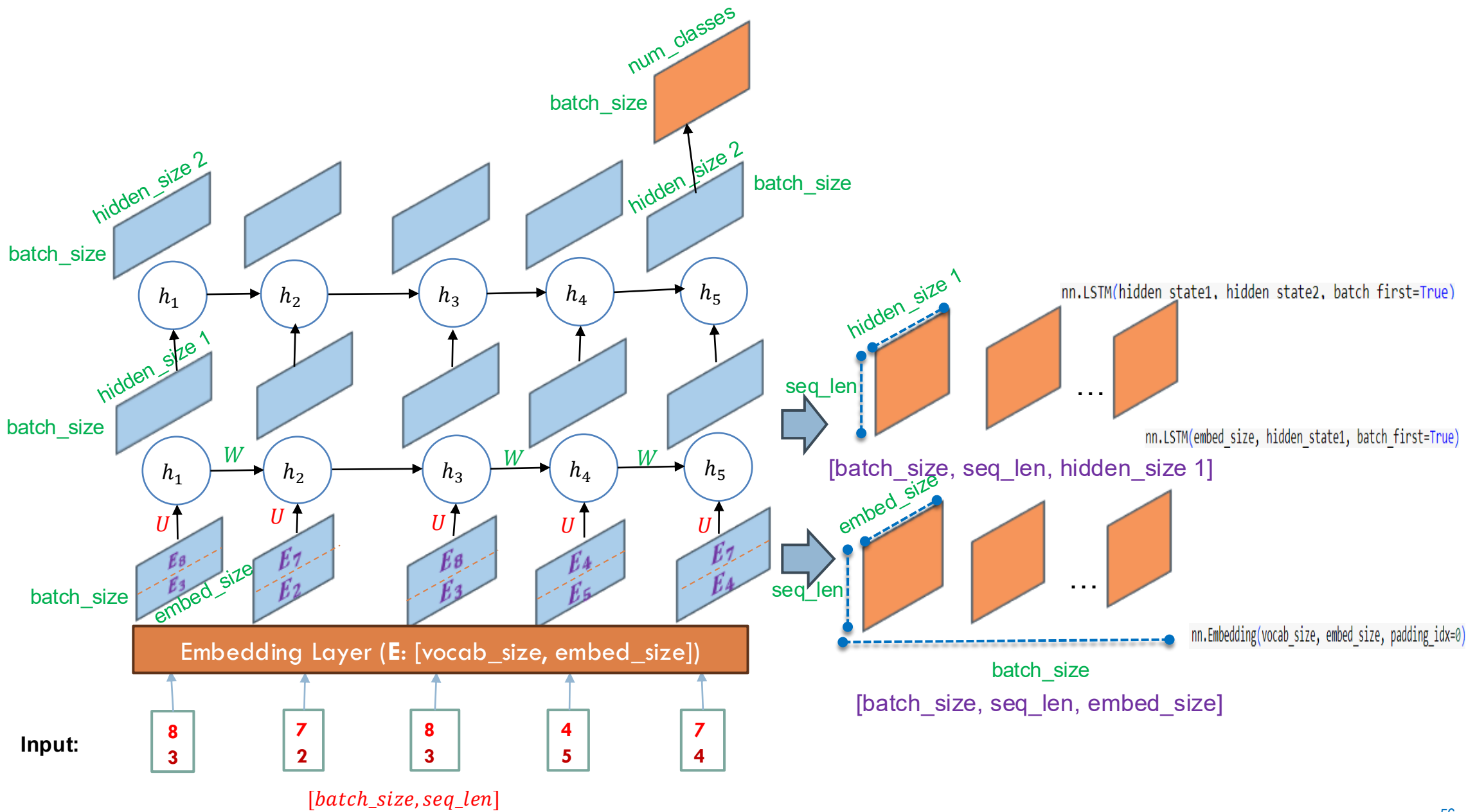
Sentiment analysis

- The word/item embedding
  - $e_1 = 1_8 E \in \mathbb{R}^{1\times 6}$
  - $e_2 = 1_7 E \in \mathbb{R}^{1\times 6}$
  - $e_3 = 1_8 E \in \mathbb{R}^{1\times 6}$
  - $e_4 = 1_4 E \in \mathbb{R}^{1\times 6}$
  - $e_5 = 1_7 E \in \mathbb{R}^{1\times 6}$
- The sequence embedding
  - $e = \begin{bmatrix} e_1 \\ e_2 \\ e_3 \\ e_4 \\ e_5 \end{bmatrix} \in \mathbb{R}^{5\times 6} = \mathbb{R}^{seq_len \times embed_size}$
- The sequence batch embedding
  - The embedding for one sequence: $\mathbb{R}^{seq_len \times embed_size}$.
  - The embedding for entire batch with $batch_size$ sequences: $\mathbb{R}^{batch_size \times seq_len \times embed_size}$





Shape Transformation in RNNs



Sentiment analysis

```
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.init as init

embed_size = 128

class Model(nn.Module):
    def __init__(self, vocab_size):
        super(Model, self).__init__()
        self.embedding = nn.Embedding(vocab_size, embed_size, padding_idx=0)

        self.lstm1 = nn.LSTM(embed_size, 128, batch_first=True)
        self.lstm2 = nn.LSTM(128, 128, batch_first=True)

        self.fc = nn.Linear(128, 1)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        x = self.embedding(x)
        x,(_,_) = self.lstm1(x)
        _,(x,_) = self.lstm2(x)
        x = self.fc(x.view(-1,128))
        return x.squeeze(1)
```

(batch_size, seq_len)

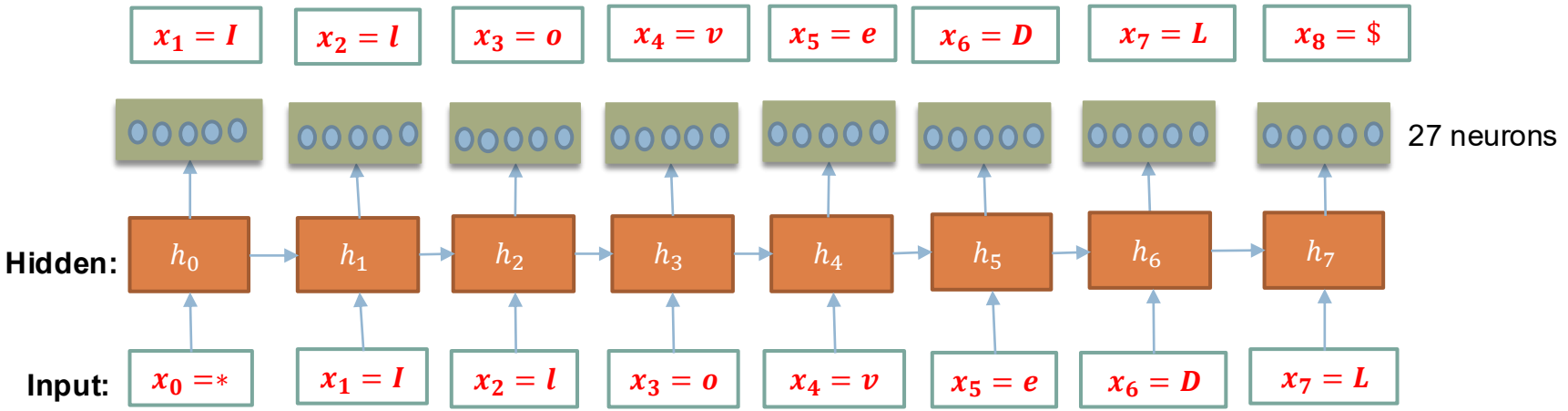
(batch_size, seq_len, embed_size)

(batch_size, seq_len, 128)

(batch_size, 128)

(batch_size, 1)

Text generation at the character level



- Consider a sentence: x="I love DL" in **our dataset**
  - $x_0 = *, x_1 = I, x_2 = l, x_3 = o, x_4 = v, x_5 = e, x_6 = D, x_7 = L, x_8 = \$$
- The joint distribution
  - $p(x_0, x_1, x_2, \dots, x_7, x_8) = p(x_0)p(x_1 \mid x_0)p(x_2 \mid x_{0:1})p(x_3 \mid x_{0:2}) \dots p(x_7 \mid x_{0:6})p(x_8 \mid x_{0:7})$
  - $p(x_0, x_1, x_2, \dots, x_7, x_8) = p(x_1 \mid h_0)p(x_2 \mid h_1)p(x_3 \mid h_2) \dots p(x_7 \mid h_6)p(x_8 \mid h_7)$
  - $\mathbf{max} \log p(x_0, x_1, x_2, \dots, x_7, x_8)$ is equivalent to $\mathbf{min}[-[\log p(x_1 \mid h_0) + \log p(x_2 \mid h_1) + \log p(x_3 \mid h_2) + \cdots + \log p(x_8 \mid h_7)]]$
  - Given h_1, we need to maximize the probability to predict x_2 and so on.
  - Cast to the problem of prediction to train our RNN
    - The loss is sum of loss at each time step

Summary

- How to define and model sequential data
- Recurrent neural networks (RNNs)
- Memory cells
  - Long-short term memory (LSTM) cell
  - Gated recurrent unit (GRU) cell
- Applications of RNNs
  - Sentiment analysis
  - Text generation

Thanks for your attention!

